

# A Zeroth-Order Paradigm for LLM Preference Alignment

**Peter Chen**

PLLC@EECS.BERKELEY.EDU

*Department of Electrical Engineering and Computer Sciences (EECS)  
University of California, Berkeley  
Berkeley, CA 94720, USA*

**Xi Chen**

XC13@STERN.NYU.EDU

*Stern School of Business  
New York University  
New York, NY 10012, USA*

**Wotao Yin**

WOTAO.YIN@ALIBABA-INC.COM

*Decision Intelligence Lab (Seattle)  
DAMO Academy, Alibaba Group U.S.  
Bellevue, WA 98004, USA*

**Tianyi Lin**

TL3335@COLUMBIA.EDU

*Department of Industrial Engineering and Operations Research (IEOR)  
Columbia University  
New York, NY 10027, USA*

## Abstract

Direct preference alignment methods are widely used to align large language models (LLMs) with human preferences because of their computational and memory efficiency. However, likelihood displacement motivates alternative ways to extract information from preference pairs with small likelihood margins. In this paper, we propose and analyze Comparison-based Preference Optimization (ComPO), a zeroth-order alignment method based on comparison oracles. ComPO extracts directional information from these pairs without directly optimizing a differentiable preference loss on them. We establish a convergence guarantee for its basic offline scheme under smoothness, gradient sparsity, and compatibility between the oracle and a latent objective. We further introduce online ComPO, which retains the offline comparison mechanism and uses unlabeled policy generations for reverse-KL control relative to a reference policy. Following the coverage perspective of preference fine-tuning, we establish a performance guarantee for a basic constrained scheme under local coverage and in-distribution pairwise reward accuracy. Experiments on Mistral, Llama, Gemma-2, Qwen3, and Gemma-3 models demonstrate improvements over existing direct alignment methods, including length-controlled win rates, with pair-level diagnostics providing evidence consistent with mitigating likelihood displacement.

**Keywords:** Preference alignment, comparison oracles, zeroth-order optimization, KL regularization, local coverage

## 1 Introduction

Generative AI has become an increasingly important tool for building and managing intelligent systems across academia, industry, and government. Large language models (LLMs) are a core part of this progress, with strong capabilities in data organization, retrieval, reasoning, and analysis (Brown et al., 2020; Chowdhery et al., 2023; Touvron et al., 2023;

(Achiam et al., 2023; Bubeck et al., 2023). Since these models are trained on large and heterogeneous corpora, they need further alignment with human preferences so that their responses are helpful, harmless, and reliable (Bai et al., 2022). A prominent approach is *reinforcement learning from human feedback* (RLHF) (Christiano et al., 2017; Stiennon et al., 2020), which first learns a reward model from human preference pairs and then optimizes the policy using reinforcement learning. Despite its empirical success (Ziegler et al., 2019; Ouyang et al., 2022; Touvron et al., 2023; Achiam et al., 2023), RLHF requires a multi-stage training pipeline and can be expensive in memory and computation. This motivates direct alignment methods, e.g., direct preference optimization (DPO) (Rafailov et al., 2023) and its variants (Azar et al., 2024; Ethayarajh et al., 2024; Park et al., 2024; Xu et al., 2024a; Tang et al., 2024a; Meng et al., 2024; Chen et al., 2025a; Zhao et al., 2025), which directly optimize the policy using preference pairs and avoid separately training a reward model.

Direct alignment methods are appealing because of their simplicity and stability. Yet, they suffer from a critical issue known as likelihood displacement. *Likelihood displacement* refers to the counter-intuitive situation where training increases the likelihood of preferred responses relative to dispreferred ones, but decreases the absolute probability of the preferred responses, leading to “unintentional unalignment” (Pal et al., 2024; Tajwar et al., 2024; Rafailov et al., 2024b; Pang et al., 2024; Liu et al., 2024b; Yuan et al., 2025; Razin et al., 2025). For example, training a model to prefer NO over NEVER can sharply increase the likelihood of YES. Practically, this issue can harm LLM behavior by shifting probability mass to unsafe responses. When the prompt asks for steps for a terrorist organization to infiltrate a government agency, Gemma-2B-it initially generates refusal responses, while DPO training can make the model comply with the unsafe request because likelihood displacement shifts probability mass away from refusal responses; see Razin et al. (2025, Table 18). Another related issue is *verbosity*, which refers to the tendency of models fine-tuned with RLHF (Singhal et al., 2024; Kabir et al., 2024) or direct alignment methods (Park et al., 2024; Amini et al., 2024; Rafailov et al., 2024a) to generate longer responses without a corresponding improvement in quality, resulting in lower efficiency and higher consumption of hardware resources.

Recent works have suggested that likelihood displacement is related to preference pairs whose preferred and dispreferred responses are similar under model-dependent measures (Pal et al., 2024; Razin et al., 2025). We refer to such small margin pairs as *noisy preference pairs* in this paper (see Eq. (10)). Existing methods have tried to mitigate likelihood displacement by adding additional regularization (Pal et al., 2024; Rafailov et al., 2024b). More recently, Razin et al. (2025) proposed to measure the similarity between preferred and dispreferred responses using the centered hidden embedding similarity (CHES) score, and empirically showed that filtering out preference pairs identified by the CHES score as problematic can be more effective for mitigating likelihood displacement than adding supervised fine-tuning (SFT) regularization. This finding highlights the role of data geometry in direct alignment. However, filtering noisy pairs also removes them from training entirely, even though these pairs may still contain useful comparative information.

While DPO provides a computationally convenient framework by maximizing a certain log-likelihood margin between preferred and dispreferred responses, this objective function can be viewed as a *proxy* for the true goal of alignment. This proxy is effective when preference pairs clearly distinguish better responses from worse responses. However, when

faced with noisy pairs – *where the preference signal is weak or ambiguous under model-based similarity measures* – optimizing a fixed DPO-style objective can lead to adverse effects such as likelihood displacement. In such cases, the pair may still provide useful local information, even if it is not suitable for direct optimization by a margin-based loss. This motivates a comparison-oracle view of preference alignment. Explicitly defining alignment as a single optimizable mathematical objective function is exceptionally challenging. Instead of pursuing such an explicit objective, we ask whether a nearby policy perturbation improves the local behavior of the model on preference pairs. A favorable perturbation should increase the likelihood of the preferred response and decrease the likelihood of the dispreferred response. In this way, noisy preference pairs are treated as comparison signals about a latent alignment objective, rather than as direct samples for a fixed loss function.

In this paper, we propose a zeroth-order preference alignment method based on comparison oracles, called ComPO. Our approach perturbs the current policy, evaluates whether each perturbation increases the likelihood of preferred responses and decreases that of dispreferred responses, and aggregates the resulting one-bit signals to estimate a normalized update direction. This allows low-margin pairs, designated as noisy, to contribute to alignment without directly optimizing a differentiable preference loss on them, complementing standard direct alignment methods applied to clean pairs. We further extend ComPO to control policy deviation using unlabeled online generations, while retaining offline preference pairs as the source of comparison signals. The motivation follows the coverage perspective of Song et al. (2024b): reverse KL can be estimated from generations of the policy being evaluated, and constraining it permits a performance analysis based on the coverage within a prescribed neighborhood of the reference policy rather than over the full policy class. Coverage remains a separate assumption and is not implied by the KL constraint.

Our empirical also examines length-related effects which have been studied in the literature (Gao et al., 2023; Dubois et al., 2023; Park et al., 2024; Amini et al., 2024; Xu et al., 2024a; Meng et al., 2024; Pang et al., 2024). Although ComPO is not specifically designed to control verbosity, we evaluate its length-controlled (LC) win rates and examine pair-level likelihood changes. We interpret higher LC win rates as improved judged performance after adjustment for response length, rather than direct evidence of shorter responses.

**Contributions.** Our contributions can be summarized as follows:

1. We develop ComPO, a comparison-based method that uses low-margin preference pairs to refine an aligned policy without directly optimizing a differentiable preference loss on those pairs. Its practical offline implementation uses output-layer perturbations and entry-wise thresholding. The online extension retains the same comparison mechanism and uses unlabeled current-policy generations to adapt the step size.
2. We establish a best-iterate convergence guarantee for the basic offline scheme under smoothness, gradient sparsity, and oracle compatibility. For the basic online scheme, we prove feasibility under an exact reverse-KL constraint and bound the performance gap in terms of in-distribution pairwise reward error under local coverage.
3. We evaluate ComPO on base and instruction-tuned models from the Mistral, Llama, Gemma-2, Qwen3, and Gemma-3 families. The experiments assess its compatibility

with direct alignment methods, its design choices, and the effects of online damping and replay. Pair-level likelihood diagnostics complement the benchmark evaluations.

**Relationship to the conference version.** A preliminary version of this work appeared at NeurIPS 2025 (Chen et al., 2025b). It introduced offline ComPO, preference comparison oracle, the convergence analysis, and the original offline experiments. The journal extension adds the online extension, its coverage-based analysis, and experiments on additional model families, including evaluations of online regularization and replay.

**Related works.** Direct preference alignment methods, including DPO (Rafailov et al., 2023), are simple and more stable offline alternatives to RLHF. Several DPO variants with alternative objectives have been proposed, including ranking-based variants beyond pairwise preference data (Dong et al., 2023; Yuan et al., 2023; Song et al., 2024a; Chen et al., 2024; Liu et al., 2025) and reference-model-free variants (Hong et al., 2024; Meng et al., 2024). It is well known that DPO suffers from the issues of verbosity (Park et al., 2024; Amini et al., 2024; Rafailov et al., 2024a) and likelihood displacement (Pal et al., 2024; Tajwar et al., 2024; Rafailov et al., 2024b; Pang et al., 2024; Liu et al., 2024b; Yuan et al., 2025), which can be interpreted from a unified perspective of data curation (Park et al., 2024; Razin et al., 2025). Our work continues along this perspective by arguing that these issues can be mitigated by using the information contained in noisy preference pairs for which the reference model assigns similar likelihoods to preferred and dispreferred responses.

Recent work has examined different roles of online data in preference fine-tuning. Online preference optimization can acquire additional labels for responses generated by the current policy, as in the online AI feedback approach of Guo et al. (2024). In contrast, Song et al. (2024b) introduce HyPO, which combines offline preference optimization with reverse-KL regularization estimated from unlabeled online samples. Our online extension follows this separation between preference supervision and regularization, but uses comparison-derived update directions. A complementary line of work studies active exploration (Xie et al., 2025) by augmenting online DPO with an explicit exploration bonus to guide the acquisition of preference feedback. Online ComPO does not acquire new preference labels or introduce such a bonus and its analysis concerns policy performance under local coverage.

Comparison-based optimization includes coordinate-search methods (Jamieson et al., 2012; Matsui et al., 2017) and directional estimators such as SCOBO (Cai et al., 2022a) and Sign-OPT (Cheng et al., 2020). Sign-OPT also provides a stationarity analysis under smoothness and additional assumptions on gradient noise, so nonconvexity alone is not the distinction from that work. ComPO specializes the comparison mechanism to preference alignment: its oracle evaluates preferred- and dispreferred-response likelihood changes, its basic analysis exploits approximately sparse gradients, and its practical implementation uses output-layer perturbations and thresholding. Comparison and ranking feedback have also been studied in bandit optimization (Yue and Joachims, 2009; Kumagai, 2017; Ding and Zhou, 2018), Bayesian optimization (Astudillo and Frazier, 2020; Lin et al., 2022b), and RLHF (Tang et al., 2024b; Zhang and Ying, 2025). Our focus is on extracting comparison signals from low-margin offline preference pairs and combining them with unlabeled online generations for step-size control.

## 2 Preliminaries

We provide an overview of the setup for direct preference alignment, and recall the definition of comparison oracles and the subroutine for estimating gradients using comparison oracles that are important for designing the basic scheme of our method. We further introduce the reverse-KL and coverage notation used in online ComPO.

### 2.1 Direct preference alignment

Modern LLMs are designed based on the Transformer architecture (Vaswani et al., 2017) and follow user prompts  $\mathbf{x} \in \mathcal{V}^*$  to generate responses  $\mathbf{y} \in \mathcal{V}^*$ , where  $\mathcal{V}$  is a vocabulary of tokens. We view an LLM as a policy  $\pi_\theta(\mathbf{y}|\mathbf{x})$  which assigns probabilities to responses  $\mathbf{y}$  given prompts  $\mathbf{x}$ . To assign probabilities to each token of  $\mathbf{y}$ , the policy  $\pi_\theta$  operates in an auto-regressive manner as follows,

$$\pi_\theta(\mathbf{y}|\mathbf{x}) = \prod_{k=1}^{|\mathbf{y}|} \pi_\theta(\mathbf{y}_k|\mathbf{x}, \mathbf{y}_{<k}),$$

where  $\theta$  denotes the model parameters (e.g., the parameters of the Transformer architecture) and  $\mathbf{y}_{<k}$  denotes the first  $k-1$  tokens of  $\mathbf{y}$ . However, the generations might not be helpful, safe, or reliable, which motivates further alignment of LLMs with human preferences.

We consider the direct preference learning pipeline based on pairwise preference data. Specifically, we assume access to a preference dataset  $D$  containing samples  $(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-)$ , where  $\mathbf{x}$  is a prompt and  $(\mathbf{y}^+, \mathbf{y}^-)$  is a pair of preferred and dispreferred responses to  $\mathbf{x}$ . This pipeline usually includes an initial supervised fine-tuning (SFT) phase, where the model is fine-tuned using the cross-entropy loss and high-quality data for specific downstream tasks. The SFT data can be either independent of  $D$  (Touvron et al., 2023), or may consist of prompts and preferred responses from  $D$  (Rafailov et al., 2023).

Direct alignment methods, such as DPO (Rafailov et al., 2023), optimize the policy  $\pi_\theta$  over the preference dataset  $D$  without learning a reward model as in RLHF (Ziegler et al., 2019; Stiennon et al., 2020). This is done by minimizing a contrastive loss as follows,

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-) \sim D} \left[ \log \sigma \left( \beta \log \frac{\pi_\theta(\mathbf{y}^+|\mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}^+|\mathbf{x})} - \beta \log \frac{\pi_\theta(\mathbf{y}^-|\mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}^-|\mathbf{x})} \right) \right], \quad (1)$$

where  $\pi_{\text{ref}}$  is the model after SFT,  $\beta$  is a regularization parameter, and  $\sigma : \mathbb{R} \rightarrow [0, 1]$  is the sigmoid function. The function  $\mathcal{L}_{\text{DPO}}$  relies on the log-likelihood margin between  $\mathbf{y}^+$  and  $\mathbf{y}^-$ . Thus, DPO improves the relative likelihood margin between the two responses, rather than directly maximizing the likelihood of  $\mathbf{y}^+$  and minimizing the likelihood of  $\mathbf{y}^-$ . During training, the likelihood of  $\mathbf{y}^+$  might decrease, and probability mass can be shifted from  $\mathbf{y}^+$  to responses with an opposite meaning (Pal et al., 2024; Razin et al., 2025). A possible reason is that the above objective function is not well suited for extracting information from noisy preference pairs whose preferred and dispreferred responses have small likelihood margins or are similar under model-based measures.

Empirically, Razin et al. (2025) show that filtering out similar preference pairs can make DPO more effective. However, noisy preference pairs might still contain useful information that can improve the performance of LLMs. Extracting such information is challenging

using a fixed margin-based loss, since maximizing the likelihood of  $\mathbf{y}^+$  and minimizing the likelihood of  $\mathbf{y}^-$  locally does not by itself define a global alignment objective. The local information we use is comparative: a better policy should assign higher likelihood to  $\mathbf{y}^+$  and lower likelihood to  $\mathbf{y}^-$ . This motivates us to design a new alignment method by directly leveraging the comparison signal in pairwise preference data  $(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-)$  from  $D$ .

## 2.2 Comparison oracles and zeroth-order methods

To contextualize our proposed method for aligning LLMs with human preferences, we review the definition of comparison oracles and explain how comparison oracles can be used to develop zeroth-order methods.

Given a function  $f : \mathbb{R}^d \rightarrow \mathbb{R}$  for which neither the function value nor the gradient is accessible, we define a pairwise comparison oracle  $\mathcal{C}_f$  in its simplest form as follows,

**Definition 2.1** *We call  $\mathcal{C}_f(\theta, \theta') : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \{+1, -1\}$  a comparison oracle for function  $f$  if*

$$\mathcal{C}_f(\theta, \theta') = \begin{cases} -1, & \text{if } f(\theta') < f(\theta), \\ +1, & \text{otherwise.} \end{cases}$$

*In other words, when queried with  $\theta$  and  $\theta'$ , the oracle  $\mathcal{C}_f(\cdot, \cdot)$  returns  $-1$  if  $f(\theta') < f(\theta)$  and  $+1$  otherwise, with ties assigned to  $+1$ .*

The key idea behind the subroutine in Cai et al. (2022a) for estimating gradients using comparison oracles is inspired by 1-bit compressed sensing (Boufounos and Baraniuk, 2008). The goal is to recover a signal  $\mathbf{g} \in \mathbb{R}^d$  from quantized measurements  $y_i = \text{sign}(\mathbf{z}_i^\top \mathbf{g})$ , where  $\mathbf{z}_i$  is a random perturbation vector drawn from a rotationally invariant distribution. The theoretical guarantee on the required number of perturbations to obtain an approximate signal was established in Plan and Vershynin (2012) and extended in Cai et al. (2022a). Notably, for a small perturbation radius  $r > 0$ , we have

$$\mathcal{C}_f(\theta, \theta + r\mathbf{z}_i) = \text{sign}(f(\theta + r\mathbf{z}_i) - f(\theta)) \approx \text{sign}(\mathbf{z}_i^\top \nabla f(\theta)).$$

Here,  $\text{sign}(0) = +1$ . Thus, the comparison label  $y_i = \mathcal{C}_f(\theta, \theta + r\mathbf{z}_i)$  serves as an approximate one-bit measurement of  $\nabla f(\theta)$ .

Another issue is that zeroth-order comparison-based methods can suffer from dimension-dependent iteration complexity bounds (Jamieson et al., 2012). This is expected because comparison oracles are even weaker than function-value oracles. This dimension dependence can be mitigated by exploiting sparse gradient structure (Wang et al., 2018; Golovin et al., 2020; Choromanski et al., 2019; Cai et al., 2022a,b). Indeed, we say that the function  $f$  has sparse gradients if  $\|\nabla f(\theta)\|_1 \leq \sqrt{s}\|\nabla f(\theta)\|$  for all  $\theta \in \mathbb{R}^d$  and some  $s \ll d$ .

The above discussion gives the subroutine for estimating sparse gradients using comparison oracles. We generate  $m$  i.i.d. perturbation vectors, denoted by  $\{\mathbf{z}_i\}_{1 \leq i \leq m}$ , compute  $y_i = \mathcal{C}_f(\theta, \theta + r\mathbf{z}_i)$  for all  $i$ , and solve the following optimization problem:

$$\hat{\mathbf{g}} = \underset{\|\mathbf{g}\|_1 \leq \sqrt{s}, \|\mathbf{g}\| \leq 1}{\text{argmax}} \sum_{i=1}^m y_i \mathbf{z}_i^\top \mathbf{g}, \quad (2)$$

where the constraints  $\|\mathbf{g}\|_1 \leq \sqrt{s}$  and  $\|\mathbf{g}\| \leq 1$  restrict the search to an approximately sparse and normalized set.



In ComPO, the latent function  $f$  is viewed as an implicit alignment objective. Instead of assuming access to its function value or gradient, we use offline preference pairs to construct a comparison oracle: a nearby policy is considered better if it assigns a higher likelihood to the preferred response and a lower likelihood to the dispreferred response.

### 2.3 Reverse KL and local coverage

The online extension of ComPO uses unlabeled policy generations for regularization, while the comparison oracle continues to use the fixed offline preference pairs. Let  $P_{\text{on}}$  denote the prompt distribution used for online generation, and let  $\pi_{\text{ref}}$  be a fixed reference policy. For the online analysis, we consider policies with a common response support and positive probabilities on that support, and assume that the relevant expectations are finite. For any such policy  $\pi$ , we define its sequence-level reverse KL relative to the reference by

$$D_{\text{RKL}}(\pi \| \pi_{\text{ref}}) = \mathbb{E}_{\mathbf{x} \sim P_{\text{on}}} [D_{\text{KL}}(\pi(\cdot | \mathbf{x}) \| \pi_{\text{ref}}(\cdot | \mathbf{x}))] = \mathbb{E}_{\mathbf{x} \sim P_{\text{on}}, \mathbf{y} \sim \pi(\cdot | \mathbf{x})} \left[ \log \frac{\pi(\mathbf{y} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y} | \mathbf{x})} \right], \quad (3)$$

The reverse KL can be estimated using unlabeled generations from the current policy being evaluated. For  $\tau > 0$ , we define the reverse-KL neighborhood of the reference policy by

$$\Pi_{\tau} = \{\pi : D_{\text{RKL}}(\pi \| \pi_{\text{ref}}) \leq \tau\} \quad (4)$$

We let  $r^*(\mathbf{x}, \mathbf{y})$  denote the ground-truth reward. For  $\beta > 0$ , we define the KL-regularized population objective by

$$J_{\beta}(\pi) = \mathbb{E}_{\mathbf{x} \sim P_{\text{on}}, \mathbf{y} \sim \pi(\cdot | \mathbf{x})} [r^*(\mathbf{x}, \mathbf{y})] - \beta D_{\text{RKL}}(\pi \| \pi_{\text{ref}}). \quad (5)$$

For a policy  $\pi$ , we define its implicit reward relative to  $\pi_{\text{ref}}$  by

$$\hat{r}_{\pi}(\mathbf{x}, \mathbf{y}) = \beta \log \frac{\pi(\mathbf{y} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y} | \mathbf{x})}. \quad (6)$$

Pairwise reward differences are invariant to prompt-dependent additive constants. We thus measure the accuracy through the following in-distribution pairwise error, where  $\mathbf{y}_1$  and  $\mathbf{y}_2$  are drawn independently from  $\pi_{\text{ref}}(\cdot | \mathbf{x})$  conditional on  $\mathbf{x}$ . Formally, we have

$$\text{err}(\pi) = \mathbb{E}_{\mathbf{x} \sim P_{\text{on}}, \mathbf{y}_1, \mathbf{y}_2 \sim \pi_{\text{ref}}(\cdot | \mathbf{x})} \left[ (r^*(\mathbf{x}, \mathbf{y}_1) - r^*(\mathbf{x}, \mathbf{y}_2) - \hat{r}_{\pi}(\mathbf{x}, \mathbf{y}_1) + \hat{r}_{\pi}(\mathbf{x}, \mathbf{y}_2))^2 \right]. \quad (7)$$

Following Song et al. (2024b), we present policy performance in terms of this in-distribution pairwise error under the local coverage condition in the following definition.

**Definition 2.2** *The reference policy  $\pi_{\text{ref}}$  satisfies local reverse-KL coverage at radius  $\kappa > 0$  with constant  $C_{\kappa} > 0$  if every policy  $\mu$  satisfying  $D_{\text{RKL}}(\mu \| \pi_{\text{ref}}) \leq \kappa$  also satisfies*

$$\sup_{\mathbf{x} \in \text{supp}(P_{\text{on}})} \sup_{\mathbf{y} \in \mathcal{Y}^*} \frac{\mu(\mathbf{y} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y} | \mathbf{x})} \leq C_{\kappa},$$

where we use the convention  $\frac{0}{0} = 0$ .

Local coverage in Definition 2.2 concerns policies within a reverse-KL neighborhood of  $\pi_{\text{ref}}$ , which guarantees that restricting the learned policy to  $\Pi_{\tau}$  can allow a performance guarantee to depend on coverage within that neighborhood. The reverse-KL constraint determines the class on which coverage is required but it does not guarantee the bounded density ratio.

### 3 Main Results

We study how to learn from noisy preference pairs that induce similar likelihoods for preferred and dispreferred responses. We first present the basic offline scheme, which replaces a first-order update driven by a predefined preference loss with a zeroth-order update driven by comparison oracles, and describe the practical offline scheme used for LLM fine-tuning. We then introduce online ComPO, which preserves the offline comparison direction and uses unlabeled current-policy generations for reverse-KL regularization.

#### 3.1 Offline preference alignment

The key idea behind ComPO is to use noisy preference pairs only to compare nearby policies. For a nonempty  $S \subseteq D$ , define

$$\begin{aligned}\Delta_S^+(\theta, \theta') &= \frac{1}{|S|} \sum_{(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-) \in S} (\log \pi_{\theta'}(\mathbf{y}^+ | \mathbf{x}) - \log \pi_{\theta}(\mathbf{y}^+ | \mathbf{x})), \\ \Delta_S^-(\theta, \theta') &= \frac{1}{|S|} \sum_{(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-) \in S} (\log \pi_{\theta'}(\mathbf{y}^- | \mathbf{x}) - \log \pi_{\theta}(\mathbf{y}^- | \mathbf{x})).\end{aligned}\tag{8}$$

We then provide the formulation of preference comparison oracle for LLM alignment below:

**Definition 3.1 (Preference comparison oracle)** *For a set  $S \subseteq D$ , the preference comparison oracle  $\mathcal{C}_{\pi}^S(\theta, \theta') : \mathbb{R}^d \times \mathbb{R}^d \mapsto \{+1, -1\}$  is defined by*

$$\mathcal{C}_{\pi}^S(\theta, \theta') = \begin{cases} -1, & \text{if } \Delta_S^+(\theta, \theta') > 0 \text{ and } \Delta_S^-(\theta, \theta') < 0, \\ +1, & \text{otherwise.} \end{cases}$$

Thus,  $\mathcal{C}_{\pi}^S(\theta, \theta') = -1$  means that  $\theta'$  is preferred to  $\theta$  according to the likelihood comparison induced by  $S$ . When  $S$  contains one pair, this reduces to the pairwise oracle. When  $S$  is a mini-batch, the oracle uses average preferred and dispreferred likelihood changes. Given a set of perturbations  $\{\mathbf{z}_i\}_{i=1}^m$ , ComPO queries  $y_i = \mathcal{C}_{\pi}^S(\theta_t, \theta_t + rz_i)$  for  $i = 1, \dots, m$  and applies the sparse 1-bit estimator from Eq. (2) as follows,

$$\hat{\mathbf{g}} = \underset{\|\mathbf{g}\|_1 \leq \sqrt{s}, \|\mathbf{g}\| \leq 1}{\operatorname{argmax}} \sum_{i=1}^m y_i \mathbf{z}_i^{\top} \mathbf{g}.\tag{9}$$

This is the only specialization of the comparison-oracle subroutine needed for offline ComPO.

The following theorem establishes a best-iterate convergence guarantee for the basic offline scheme under smoothness, gradient sparsity, and oracle compatibility.

**Theorem 3.2** *Fix a nonempty comparison set  $S \subseteq D$  and  $1 \leq s \leq d$ . Suppose that there exists an  $\ell$ -smooth function  $f : \mathbb{R}^d \rightarrow \mathbb{R}$ , with  $\ell > 0$ , that is bounded below and satisfies*

1. *For all  $(\theta, \theta')$ , we have  $\mathcal{C}_{\pi}^S(\theta, \theta') = -1$  if  $f(\theta') < f(\theta)$  and  $\mathcal{C}_{\pi}^S(\theta, \theta') = 1$  otherwise.*
2. *The gradients of  $f$  are approximately sparse:  $\|\nabla f(\theta)\|_1 \leq \sqrt{s} \|\nabla f(\theta)\|$  for all  $\theta \in \mathbb{R}^d$ .*

*Let  $\Delta > 0$  satisfy  $f(\theta_1) - \inf_{\theta \in \mathbb{R}^d} f(\theta) \leq \Delta$ . For any  $\epsilon, \Lambda \in (0, 1)$ , we choose*

$$T = \lceil \frac{10\ell\Delta}{\epsilon^2} \rceil, \quad \eta = \sqrt{\frac{2\Delta}{\ell T}}, \quad r = \frac{\epsilon}{40\ell\sqrt{d}}, \quad m = \lceil c_m \left( s \log \left( \frac{2d}{s} \right) + \log \left( \frac{2T}{\Lambda} \right) \right) \rceil,$$



**Algorithm 1** Offline ComPO: Basic Scheme

- 
- 1: **Input:** initial parameter  $\theta_1 \in \mathbb{R}^d$ , comparison set  $S \subseteq D$ , step size  $\eta > 0$ , sparsity ratio  $s \ll d$ , sampling radius  $r > 0$ , number of perturbations  $m \geq 1$ , and iteration number  $T \geq 1$ .
  - 2: **for**  $t = 1, 2, \dots, T$  **do**
  - 3:   Draw  $m$  i.i.d. samples uniformly from the unit sphere in  $\mathbb{R}^d$ , denoted by  $\{\mathbf{z}_i\}_{i=1}^m$ .
  - 4:   Compute  $y_i = \mathcal{C}_\pi^S(\theta_t, \theta_t + r\mathbf{z}_i)$  for  $i = 1, \dots, m$ .
  - 5:   Compute  $\hat{\mathbf{g}}_t$  using Eq. (9).
  - 6:   Update  $\theta_{t+1} = \theta_t - \eta \hat{\mathbf{g}}_t$ .
  - 7: **Output:**  $\theta_{T+1}$ .
- 

**Algorithm 2** Offline ComPO: Practical Scheme

- 
- 1: **Input:** initial parameter  $\theta_1 = [\bar{\theta}; \theta_1^o]$ , batches  $\{S_t\}_{t=1}^T$ , step size  $\gamma$ , sampling radius  $r$ , number of perturbations  $m \geq 1$ , clipping thresholds  $\lambda_g, \lambda$ , and iteration number  $T \geq 1$ .
  - 2: **for**  $t = 1, 2, \dots, T$  **do**
  - 3:   Draw  $m$  i.i.d. samples  $\{\mathbf{z}_i\}_{i=1}^m$  uniformly from the unit sphere in  $\mathbb{R}^{d_o}$ .
  - 4:   Query  $y_i = \mathcal{C}_{\pi^t}^{S_t}([\bar{\theta}; \theta_t^o], [\bar{\theta}; \theta_t^o + r\mathbf{z}_i])$  for all  $i = 1, \dots, m$ .
  - 5:   Set  $\mathbf{u}_t = \sum_{i=1}^m y_i \mathbf{z}_i$ . If  $\mathbf{u}_t \neq 0$ , set  $\hat{\mathbf{g}}_t^o = \mathbf{u}_t / \|\mathbf{u}_t\|$ . Otherwise, set  $\hat{\mathbf{g}}_t^o = 0$ .
  - 6:   Clip  $\hat{\mathbf{g}}_t^o$  by zeroing out entries whose magnitude is less than  $\lambda_g$ .
  - 7:   Set  $p_t = \frac{|\{i: y_i = -1\}|}{m}$ .
  - 8:   **if**  $p_t > \lambda$  **then**
  - 9:      $\theta_{t+1}^o = \theta_t^o - \gamma p_t \hat{\mathbf{g}}_t^o$ .
  - 10:   **else**
  - 11:      $\theta_{t+1}^o = \theta_t^o$ .
  - 12: **Output:**  $\theta_{T+1} = [\bar{\theta}; \theta_{T+1}^o]$ .
- 

where  $c_m$  is a sufficiently large constant. Suppose that the perturbations at each iteration are drawn independently of the past and Eq. (9) is solved exactly. Then, the iterates generated by Algorithm 1 satisfy

$$\mathbb{P} \left( \min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| < \epsilon \right) \geq 1 - \Lambda.$$

Consequently, the total number of preference-comparison oracle calls is bounded by

$$O \left( \left( 1 + \frac{\ell \Delta}{\epsilon^2} \right) \left( s \log \left( \frac{2d}{s} \right) + \log \left( \frac{2 + \ell \Delta \epsilon^{-2}}{\Lambda} \right) \right) \right).$$

**Remark 3.3** Theorem 3.2 provides a best-iterate convergence guarantee for the basic offline scheme under the stated assumptions. Since the objective  $f$  is latent, its gradient norm is not available as a practical stopping criterion. The result nevertheless provides a theoretical benchmark: for fixed sparsity level  $s$ , the number of comparison queries depends only logarithmically on the ambient dimension. The practical implementation below approximates the basic estimator to accommodate the scale of LLM fine-tuning.

**Practical scheme.** Applying the basic scheme to all model parameters is computationally expensive for LLMs. We therefore perturb only the output-layer weights  $\theta^o \in \mathbb{R}^{d_o}$  and freeze the remaining parameters  $\bar{\theta}$ , so that  $\theta = [\bar{\theta}; \theta^o]$ . We also replace the exact solution of Eq. (9) with a normalized sum of signed perturbations followed by entry-wise clipping.

**Algorithm 3** Online ComPO: Basic Scheme

- 
- 1: **Input:** initial parameter  $\theta_1 \in \mathbb{R}^d$  satisfying  $\pi_{\theta_1} \in \Pi_\tau$ , comparison set  $S \subseteq D$ , online prompt distribution  $P_{\text{on}}$ , reference policy  $\pi_{\text{ref}}$ , step size  $\eta > 0$ , reverse-KL radius  $\tau > 0$ , sparsity ratio  $s \ll d$ , sampling radius  $r > 0$ , number of perturbations  $m \geq 1$ , and iteration number  $T \geq 1$ .
  - 2: **for**  $t = 1, 2, \dots, T$  **do**
  - 3:   Draw  $m$  i.i.d. samples uniformly from the unit sphere in  $\mathbb{R}^d$ , denoted by  $\{\mathbf{z}_i\}_{i=1}^m$ .
  - 4:   Compute  $y_i = \mathcal{C}_\pi^S(\theta_t, \theta_t + r\mathbf{z}_i)$  for  $i = 1, \dots, m$ .
  - 5:   Compute  $\hat{\mathbf{g}}_t$  using Eq. (9).
  - 6:   Form  $\tilde{\theta}_{t+1}$  and evaluate  $\tilde{D}_t$  by Eq. (11).
  - 7:   Set  $\theta_{t+1}$  according to Eq. (12).
  - 8: **Output:**  $\theta_{T+1}$ .
- 

The practical pipeline partitions the dataset using the reference model. In particular, we define

$$D_{\text{noisy}} = \{(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-) \in D : |\log \pi_{\text{ref}}(\mathbf{y}^+|\mathbf{x}) - \log \pi_{\text{ref}}(\mathbf{y}^-|\mathbf{x})| \leq \delta_{\text{margin}}\}, \quad (10)$$

and let  $D_{\text{clean}} = D \setminus D_{\text{noisy}}$ . The term noisy refers to this low-margin subset and does not presume that its preference labels are incorrect. We first apply a direct alignment method, such as DPO or SimPO, to  $D_{\text{clean}}$  and then apply Algorithm 2 to  $D_{\text{noisy}}$ . For DPO in the first stage, we denote the resulting procedure by DPO<sub>clean</sub>+ComPO.

### 3.2 Online ComPO

We introduce an online extension of ComPO that retains the offline comparison mechanism and uses unlabeled policy generations for reverse-KL control. Following Song et al. (2024b), we restrict the policy to the class  $\Pi_\tau$  in Eq. (4), so that the analysis requires coverage only within this neighborhood. Since the update direction is obtained from comparisons rather than the gradient of an explicit preference loss, the basic scheme implements this restriction through a feasibility check on each candidate update. The practical scheme uses the samples from the current policy to adjust the step size.

At iteration  $t$ , we compute the same comparison direction  $\hat{\mathbf{g}}_t$  as in Algorithm 1 and form a single candidate using a fixed step size  $\eta > 0$ :

$$\tilde{\theta}_{t+1} = \theta_t - \eta \hat{\mathbf{g}}_t, \quad \tilde{D}_t = D_{\text{RKL}}(\pi_{\tilde{\theta}_{t+1}} \parallel \pi_{\text{ref}}). \quad (11)$$

Given a reverse-KL radius  $\tau > 0$ , we accept the candidate if it is feasible and otherwise leave the policy unchanged:

$$\theta_{t+1} = \begin{cases} \tilde{\theta}_{t+1}, & \text{if } \tilde{D}_t \leq \tau, \\ \theta_t, & \text{otherwise.} \end{cases} \quad (12)$$

The basic scheme evaluates the candidate policy’s reverse KL exactly. Starting from a feasible policy, the accept-or-reject rule preserves feasibility by retaining the previous iterate whenever the candidate falls outside  $\Pi_\tau$ .

The following theorem establishes feasibility and relates in-distribution pairwise reward accuracy to policy performance under local coverage.

**Algorithm 4** Online ComPO: Practical Scheme

---

```

1: Input: initial parameter  $\theta_1 = [\bar{\theta}; \theta_1^o]$ , preference dataset  $D$ , online prompts  $\mathcal{X}_{\text{on}}$ , reference policy
    $\pi_{\text{ref}}$ , margin threshold  $\delta_{\text{margin}}$ , step-size scale  $\gamma > 0$ , damping strength  $\rho \geq 0$ , threshold  $\tau_p \geq 0$ ,
   sampling radius  $r > 0$ , number of perturbations  $m \geq 1$ , online batch size  $B \geq 1$ , clipping
   thresholds  $\lambda_g, \lambda > 0$ , iteration number  $T \geq 1$ , re-sampling window  $n \geq 1$ , and replay ratio
    $\alpha \in [0, 1]$ .
2: Construct  $D_{\text{noisy}}$  using Eq. (10).
3: Initialize the replay buffer  $\mathcal{R} \leftarrow \emptyset$  and the current successful-batch buffer  $\mathcal{A} \leftarrow \emptyset$ .
4: for  $t = 1, 2, \dots, T$  do
5:   if  $t > 1$  and  $(t-1) \bmod n = 0$  then
6:     Set  $\mathcal{R} \leftarrow \mathcal{A}$  and  $\mathcal{A} \leftarrow \emptyset$ .
7:   Draw a replay indicator  $b_t \sim \text{Bernoulli}(\alpha)$ .
8:   if  $b_t = 1$  and  $\mathcal{R} \neq \emptyset$  then
9:     Sample a previously successful preference mini-batch  $S_t$  uniformly from  $\mathcal{R}$ .
10:  else
11:    Sample a new noisy preference mini-batch  $S_t \subseteq D_{\text{noisy}}$ .
12:    Draw  $m$  i.i.d. samples uniformly from the unit sphere in  $\mathbb{R}^{d_o}$ , denoted by  $\{\mathbf{z}_i\}_{i=1}^m$ .
13:    Query  $y_i = C_{\pi}^{S_t}([\bar{\theta}; \theta_t^o], [\bar{\theta}; \theta_t^o + r\mathbf{z}_i])$  for  $i = 1, \dots, m$ .
14:    Set  $\mathbf{u}_t = \sum_{i=1}^m y_i \mathbf{z}_i$  and  $\hat{\mathbf{g}}_t^o = \mathbf{u}_t / \|\mathbf{u}_t\|$  if  $\mathbf{u}_t \neq 0$ ; otherwise set  $\hat{\mathbf{g}}_t^o = 0$ . Clip  $\hat{\mathbf{g}}_t^o$  by zeroing out
       entries whose magnitude is less than  $\lambda_g$ .
15:    Sample  $\{\tilde{\mathbf{x}}_j\}_{j=1}^B \subseteq \mathcal{X}_{\text{on}}$ , generate  $\tilde{\mathbf{y}}_j \sim \pi_{\theta_t}(\cdot | \tilde{\mathbf{x}}_j)$ , and compute  $\hat{d}_t$  and  $\gamma_t$  using Eq. (14)-(15).
16:    Set  $p_t = \frac{|\{i: y_i = -1\}|}{m}$ .
17:    if  $p_t > \lambda$  then
18:       $\theta_{t+1}^o = \theta_t^o - \gamma_t p_t \hat{\mathbf{g}}_t^o$ .
19:      Add the accepted preference mini-batch to the current buffer:  $\mathcal{A} \leftarrow \mathcal{A} \cup \{S_t\}$ .
20:    else
21:       $\theta_{t+1}^o = \theta_t^o$ .
22: Output:  $\theta_{T+1} = [\bar{\theta}; \theta_{T+1}^o]$ .

```

---

**Theorem 3.4** Fix  $\beta, \tau > 0$ . Suppose that Algorithm 3 evaluates each candidate policy’s reverse KL exactly. Then, the generated iterates satisfy  $\pi_{\theta_t} \in \Pi_\tau$  for all  $t = 1, \dots, T+1$ . If  $\pi_{\text{ref}}$  satisfies local reverse-KL coverage at radius  $\tau$  with constant  $C_\tau$ , we have

$$\sup_{\pi \in \Pi_\tau} J_\beta(\pi) - J_\beta(\pi_{\theta_t}) \leq C_\tau \sqrt{\text{err}(\pi_{\theta_t})}, \quad \text{for all } t = 1, \dots, T+1.$$

For any  $\epsilon > 0$ , an iterate satisfying  $\text{err}(\pi_{\theta_t}) \leq \epsilon$  satisfies  $\sup_{\pi \in \Pi_\tau} J_\beta(\pi) - J_\beta(\pi_{\theta_t}) \leq C_\tau \sqrt{\epsilon}$ .

Theorem 3.4 combines the feasibility preservation with a coverage-based performance bound following Song et al. (2024b). The reverse-KL constraint restricts the policies under consideration to  $\Pi_\tau$ , so that this guarantee requires coverage within the neighborhood rather than over the entire policy class. Within this neighborhood, smaller pairwise reward error gives a tighter performance bound.

**Practical scheme.** While the basic scheme evaluates reverse KL at the candidate policy, the practical scheme samples from the current policy and uses a length-normalized statistic to damp the update. For independent prompts  $\tilde{\mathbf{x}}_j \sim P_{\text{on}}$  and responses  $\tilde{\mathbf{y}}_j \sim \pi_{\theta_t}(\cdot | \tilde{\mathbf{x}}_j)$ , the

sequence-level estimator

$$\widehat{D}_t^{\text{seq}} = \frac{1}{B} \sum_{j=1}^B \log \left( \frac{\pi_{\theta_t}(\tilde{\mathbf{y}}_j | \tilde{\mathbf{x}}_j)}{\pi_{\text{ref}}(\tilde{\mathbf{y}}_j | \tilde{\mathbf{x}}_j)} \right) \quad (13)$$

is unbiased for  $D_{\text{RKL}}(\pi_{\theta_t} \parallel \pi_{\text{ref}})$ . In practice, we use

$$\hat{d}_t = \frac{1}{B} \sum_{j=1}^B \frac{\log \pi_{\theta_t}(\tilde{\mathbf{y}}_j | \tilde{\mathbf{x}}_j) - \log \pi_{\text{ref}}(\tilde{\mathbf{y}}_j | \tilde{\mathbf{x}}_j)}{\max\{1, |\tilde{\mathbf{y}}_j|\}}. \quad (14)$$

Length normalization changes the population quantity being estimated. In particular,  $\hat{d}_t$  is a signed statistic and its population counterpart needs not be nonnegative. We set

$$\gamma_t = \frac{\gamma}{1 + \rho \max\{\hat{d}_t - \tau_p, 0\}}, \quad (15)$$

where  $\tau_p$  is the threshold for the length-normalized statistic. As such, the online samples only change the step size, not the comparison oracle or the preference labels.

We divide training into consecutive blocks of  $n$  iterations. At the start of each block after the first, the replay buffer is replaced by the mini-batches that passed the update gate  $p_t > \lambda$  in the preceding completed block. At each iteration, with probability  $\alpha$ , we sample uniformly from this buffer when it is nonempty; otherwise, we sample a new mini-batch from  $D_{\text{noisy}}$ . Revisited mini-batches use fresh perturbations around the current parameters rather than reusing previous update directions. Here, “successful” means only that the comparison gate was passed. Section 4.3 evaluates the empirical effect of combining replay with online damping.

Algorithm 4 is motivated by the principle used in Algorithm 3, but cannot be covered by Theorem 3.4. In particular, the length-normalized quantity in Eq. (14) is not the sequence-level reverse KL in Eq. (3), and Eq. (15) does not enforce the hard constraint  $\pi \in \Pi_{\tau}$ . These are practical heuristics whose effect is evaluated empirically in Section 4.3.

## 4 Experiments

We investigate the effectiveness of ComPO on aligning the LLMs. First, we evaluate offline scheme as an augmentation to DPO and its variants, where it extracts the directions from noisy preference pairs. Second, we study the offline design choices and the scaling behavior with respect to perturbations, perturbed layers and noisy pairs. Third, we evaluate online scheme, which uses unlabeled current-policy generations to damp the step through a reverse-KL proxy. Unless otherwise stated, the main tables report point estimates from the reported runs and the ablation tables explicitly report variation across repeated runs.

### 4.1 Offline training for augmenting DPO and SimPO

We identify clean and noisy preference pairs using the margin threshold  $\delta_{\text{margin}} = 3$ . For Mistral-7B models, we set  $r = 0.0005$ ,  $m = 1600$ ,  $\lambda_g = 0.00022$ , and  $\lambda = 0.2$ . For Llama-3-8B models and Gemma-2-9B-it, we set  $r = 0.00075$ ,  $m = 1800$ ,  $\lambda_g = 0.00008$ , and  $\lambda = 0.2$ . We use UltraFeedback<sup>1</sup> (Cui et al., 2024) throughout the offline experiments. We initialize

1. [https://huggingface.co/datasets/HuggingFaceH4/ultrafeedback\\_binarized](https://huggingface.co/datasets/HuggingFaceH4/ultrafeedback_binarized)

Table 1: Evaluation on AlpacaEval 2, Arena-Hard, and MT-Bench across four model configurations. LC and WR denote length-controlled win rate and raw win rate, respectively. Turn-1 and Turn-2 are the MT-Bench scores for the initial and follow-up questions. “PA” denotes the pre-alignment supervised or instruction-fine-tuned checkpoint before DPO training.

| Method                      | Mistral-7B-Base |             |            |             |             |             | Mistral-7B-Instruct |              |             |             |             |             |
|-----------------------------|-----------------|-------------|------------|-------------|-------------|-------------|---------------------|--------------|-------------|-------------|-------------|-------------|
|                             | AlpacaEval 2    |             | Arena-Hard | MT-Bench    |             |             | AlpacaEval 2        |              | Arena-Hard  | MT-Bench    |             |             |
|                             | LC (%)          | WR (%)      | WR (%)     | Turn-1      | Turn-2      | Avg.        | LC (%)              | WR (%)       | WR (%)      | Turn-1      | Turn-2      | Avg.        |
| PA                          | 7.33            | 4.48        | 1.1        | 6.10        | 5.04        | 5.57        | 16.54               | 12.43        | 10.9        | 6.19        | 5.10        | 5.65        |
| DPO                         | 9.71            | 6.27        | 2.9        | 6.20        | <b>5.38</b> | <b>5.79</b> | 24.14               | 16.71        | <b>14.4</b> | 6.28        | 5.42        | 5.86        |
| DPO <sub>clean</sub>        | 9.41            | 6.52        | 3.0        | 6.18        | 5.22        | 5.70        | 23.89               | 16.15        | 14.2        | 6.11        | 5.34        | 5.73        |
| DPO <sub>clean</sub> +ComPO | <b>11.66</b>    | <b>6.55</b> | <b>3.2</b> | <b>6.22</b> | 5.32        | 5.77        | <b>26.17</b>        | <b>18.32</b> | 10.5        | <b>7.78</b> | <b>7.63</b> | <b>7.69</b> |

| Method                      | Llama-3-8B-Base |              |             |             |             |             | Llama-3-8B-Instruct |              |             |             |             |             |
|-----------------------------|-----------------|--------------|-------------|-------------|-------------|-------------|---------------------|--------------|-------------|-------------|-------------|-------------|
|                             | AlpacaEval 2    |              | Arena-Hard  | MT-Bench    |             |             | AlpacaEval 2        |              | Arena-Hard  | MT-Bench    |             |             |
|                             | LC (%)          | WR (%)       | WR (%)      | Turn-1      | Turn-2      | Avg.        | LC (%)              | WR (%)       | WR (%)      | Turn-1      | Turn-2      | Avg.        |
| PA                          | 3.21            | 7.97         | 4.1         | 6.53        | 5.66        | 6.10        | 24.06               | 23.69        | 20.8        | 8.22        | 7.57        | 7.90        |
| DPO                         | 4.14            | 10.43        | <b>12.1</b> | 6.61        | 5.85        | 6.23        | 32.59               | 31.99        | 22.9        | 8.30        | 7.55        | 7.93        |
| DPO <sub>clean</sub>        | 4.28            | 9.81         | 12.0        | <b>6.64</b> | 6.01        | 6.33        | 32.92               | 32.42        | 22.9        | 8.26        | 7.63        | 7.94        |
| DPO <sub>clean</sub> +ComPO | <b>5.39</b>     | <b>10.93</b> | <b>12.1</b> | 6.60        | <b>6.28</b> | <b>6.44</b> | <b>35.79</b>        | <b>35.03</b> | <b>23.1</b> | <b>8.39</b> | <b>7.71</b> | <b>8.05</b> |

from the supervised fine-tuned **Base** and **Instruct** models used in Meng et al. (2024): Mistral-7B Base and Instruct<sup>2</sup>, Llama-3-8B Base<sup>3</sup> and Instruct<sup>4</sup>, and Gemma-2-9B-it<sup>5</sup>. All ComPO runs use 30 NVIDIA A40 GPUs, each with 46 GB of memory.

We follow the evaluation protocol of Meng et al. (2024) and evaluate on AlpacaEval 2-v0.6.6 (Li et al., 2023), Arena-Hard (Li et al., 2024), and MT-Bench (Zheng et al., 2023). For AlpacaEval 2, GPT-4 Turbo serves as both baseline and judge models. The judge compares each model response with the baseline response, and we report raw win rate (WR) and length-controlled win rate (LC) (Dubois et al., 2024). LC adjusts judged preferences for response length and a higher LC score does not by itself establish shorter responses. For Arena-Hard, the baseline is GPT-4-0314 and the judge is GPT-4 Turbo. We report WR. For MT-Bench, GPT-4 scores multi-turn Q&A responses on a 10-point scale. We report the scores for the initial question (Turn-1), the follow-up question (Turn-2), and their average.

**DPO with ComPO.** We split the data into clean and noisy subsets using the margin criterion in Eq. (10). Starting from the SFT model, we train on all pairs to obtain DPO and on only the clean pairs to obtain DPO<sub>clean</sub>. Following Meng et al. (2024), both models are trained for one epoch. We initialize ComPO from DPO<sub>clean</sub> and run it for one epoch with 100 iterations over noisy pairs, yielding DPO<sub>clean</sub>+ComPO.

We summarize the results in Table 1 and report three key observations. First, filtering low-margin pairs alone does not uniformly improve DPO: DPO<sub>clean</sub> is comparable to DPO overall and performs better only for some initializations, such as Llama-3-Instruct-8B. The log-likelihood margin therefore appears to be an imperfect proxy for pair ambiguity; richer

2. <https://huggingface.co/alignment-handbook/zephyr-7b-sft-full>

3. <https://huggingface.co/princeton-nlp/Llama-3-Base-8B-SFT>

4. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>

5. <https://huggingface.co/google/gemma-2-9b-it>

Table 2: Pairwise log-likelihoods in three independent trials for  $\gamma \in \{0.1, 1\}$ , with all other hyperparameters fixed at their default values. Each cell reports  $(\log \pi_\theta(\mathbf{y}^+|\mathbf{x}), \log \pi_\theta(\mathbf{y}^-|\mathbf{x}))$  after one training run; the initial values appear in the model headers. The trials use independently sampled perturbations  $\{\mathbf{z}_i\}_{1 \leq i \leq m}$ . Across the reported trials, the preferred-response log-likelihood is nondecreasing and the dispreferred-response log-likelihood is nonincreasing.

| <b>Llama-3-Instruct-8B</b> $(\log \pi_\theta(\mathbf{y}^+ \mathbf{x}), \log \pi_\theta(\mathbf{y}^- \mathbf{x})) = (-46.761, -47.410)$ |                        |                        |                        |
|----------------------------------------------------------------------------------------------------------------------------------------|------------------------|------------------------|------------------------|
| $\gamma$                                                                                                                               | Trial 1                | Trial 2                | Trial 3                |
| 0.1                                                                                                                                    | $(-46.744, -47.411)$   | $(-46.760, -47.411)$   | $(-46.759, -47.410)$   |
| 1                                                                                                                                      | $(-46.728, -47.520)$   | $(-46.743, -47.525)$   | $(-46.753, -47.517)$   |
| <b>Gemma-2-9B-it</b> $(\log \pi_\theta(\mathbf{y}^+ \mathbf{x}), \log \pi_\theta(\mathbf{y}^- \mathbf{x})) = (-133.122, -134.557)$     |                        |                        |                        |
| $\gamma$                                                                                                                               | Trial 1                | Trial 2                | Trial 3                |
| 0.1                                                                                                                                    | $(-133.122, -134.557)$ | $(-133.122, -134.557)$ | $(-133.121, -134.557)$ |
| 1                                                                                                                                      | $(-133.059, -134.562)$ | $(-133.122, -134.564)$ | $(-133.112, -134.565)$ |

criteria such as the CHES score (Razin et al., 2025) may separate pairs more accurately. Nevertheless, the margin is inexpensive to compute, and ComPO extracts useful information from the pairs that it filters out. Second, gains are especially consistent in AlpacaEval 2 LC, indicating improved judged performance after adjustment for response length. We interpret these scores separately from the response-length measurements reported below. Third, ComPO uses only the first 100 noisy pairs, yet improves most model-benchmark combinations. As such, a small set of low-margin pairs can contain useful alignment information when processed through comparison oracles.

The main exception is Arena-Hard for Mistral-7B-Instruct, where DPO scores 14.4 and  $\text{DPO}_{\text{clean}} + \text{ComPO}$  scores 10.5; for the two Llama configurations, the scores are tied or nearly tied. An explanation is that Arena-Hard reports raw rather than length-controlled win rate and can therefore favor longer generations (Meng et al., 2024). For Mistral-7B-Instruct, the average response length is 513 for DPO and 468 for  $\text{DPO}_{\text{clean}} + \text{ComPO}$ . This difference is consistent with the lower Arena-Hard score and the stronger AlpacaEval 2 LC score, although it does not by itself establish causality.

We also inspect whether the comparison oracle moves the likelihoods of each noisy pair in the intended direction. In Table 2, we summarize three independent trials for  $\gamma \in \{0.1, 1\}$  on Llama-3-Instruct-8B and Gemma-2-9B-it. For example, with Llama-3-Instruct-8B and  $\gamma = 1$ , the first trial changes the pair from  $(-46.761, -47.410)$  to  $(-46.728, -47.520)$ : the preferred response becomes more likely, while the dispreferred response becomes less likely. Thus, for the two reported models, the oracle-based update moves the pairwise likelihoods in the desired direction or leaves them unchanged. This diagnostic is an in-training sanity check rather than a population-level performance guarantee.

The thresholds  $\lambda_g$  and  $\lambda$  limit the coordinates and iterations on which the practical scheme updates the model. Very large step sizes can still destabilize the practical scheme, while Theorem 3.2 analyzes the step size only for the basic scheme. Section 4.3 considers adaptive step-size control based on current-policy generations.

**SimPO with ComPO.** ComPO is not tied to DPO. We apply it directly to existing, well-tuned SimPO checkpoints (Meng et al., 2024) and use the training and evaluation configuration described at the beginning of Section 4.1. Table 3 shows that SimPO+ComPO



Table 3: Applying ComPO to existing SimPO checkpoints across models and benchmarks.

| Model                      | Method        | AlpacaEval 2 |              | Arena-Hard  | MT-Bench    |             |             |
|----------------------------|---------------|--------------|--------------|-------------|-------------|-------------|-------------|
|                            |               | LC (%)       | WR (%)       | WR (%)      | Turn-1      | Turn-2      | Avg.        |
| <b>Mistral-7B-Instruct</b> | SimPO         | 40.22        | 41.18        | 20.8        | <b>7.94</b> | 7.31        | 7.62        |
|                            | SimPO + ComPO | <b>42.27</b> | <b>43.17</b> | <b>22.0</b> | 7.83        | <b>7.46</b> | <b>7.64</b> |
| <b>Llama-3-8B-Instruct</b> | SimPO         | 48.71        | 43.66        | 36.3        | 7.91        | 7.42        | 7.66        |
|                            | SimPO + ComPO | <b>49.53</b> | <b>45.03</b> | <b>37.3</b> | <b>7.94</b> | <b>7.45</b> | <b>7.70</b> |
| <b>Gemma-2-9B-it</b>       | SimPO         | 60.36        | 55.59        | <b>61.1</b> | <b>9.07</b> | 8.47        | 8.77        |
|                            | SimPO + ComPO | <b>62.42</b> | <b>57.20</b> | <b>61.1</b> | 8.99        | <b>8.58</b> | <b>8.79</b> |

Table 4: Effect of the number of perturbations  $m$  on AlpacaEval 2. Entries are mean  $\pm$  standard deviation over five runs, with the best run in parentheses.

| Perturbation ( $m$ )     | 800                      | 1600                     | 3300                              | 5400                              |
|--------------------------|--------------------------|--------------------------|-----------------------------------|-----------------------------------|
| <b>AlpacaEval 2-WR %</b> | $17.32 \pm 0.86$ (17.94) | $17.50 \pm 0.65$ (18.32) | $19.21 \pm 0.58$ ( <b>20.25</b> ) | $19.69 \pm 0.36$ (20.07)          |
| <b>AlpacaEval 2-LC %</b> | $24.72 \pm 1.02$ (25.12) | $25.02 \pm 0.91$ (26.17) | $25.91 \pm 0.95$ (27.14)          | $26.49 \pm 0.81$ ( <b>27.20</b> ) |

improves both AlpacaEval 2 metrics for all three models. On Arena-Hard, it improves Mistral-7B-Instruct and Llama-3-8B-Instruct and matches Gemma-2-9B-it. The MT-Bench average also increases slightly for each model. These results show that ComPO augments other direct alignment methods without changing its original training objective.

## 4.2 Ablation studies

**Number of perturbations.** The number of perturbations controls how many directions the comparison oracle evaluates. We vary  $m$  while holding the remaining hyperparameters fixed and use Mistral-7B-Instruct for this study. As  $m$  increases from 800 to 5400, the mean WR and LC improve, with diminishing gains at larger  $m$  (Table 4). This is consistent with a more accurate gradient estimate from additional perturbations, although the computation time increases. Peak memory remains unchanged because ComPO accumulates a running average rather than storing all perturbation vectors (see Line 5 of Algorithm 2).

We also investigate whether ComPO scales beyond output-layer perturbations. Keeping all other settings fixed, we perturb the MLPs in layers 30–31 together with the output layer of Mistral-7B-Instruct. Table 5 uses GPT-4.1 as the Arena-Hard judge, and perturbing three layers improves all three reported metrics. The larger search space has a modest systems cost in this setup: peak GPU memory increases from 16.3 GB to 16.7 GB, and 600 perturbations take 60 seconds rather than 50 seconds.

**Gradient threshold and number of noisy pairs.** ComPO uses the entry threshold  $\lambda_g$  to update only gradient entries with sufficiently large magnitude. We vary  $\lambda_g$  with  $m = 3300$  on Mistral-7B-Instruct (Table 6). The strongest results occur when approximately 1%–6% of the entries are retained. Retaining many small entries or filtering almost all entries leads to lower performance. We then increase the number of noisy pairs from 100 to 300. Table 7 shows higher mean performance on both AlpacaEval 2 metrics and Arena-Hard, indicating that ComPO continues to benefit from additional low-margin pairs.

Table 5: Effect of perturbing multiple layers. We report AlpacaEval 2 WR and LC and Arena-Hard WR. Entries are mean  $\pm$  standard deviation over five runs, with the best run in parentheses.

| Layers perturbed (# params) | AlpacaEval 2-WR %        | AlpacaEval 2-LC %        | Arena-Hard (GPT-4.1)-WR % |
|-----------------------------|--------------------------|--------------------------|---------------------------|
| 1 (0.13B)                   | 17.50 $\pm$ 0.65 (18.32) | 25.02 $\pm$ 0.91 (26.17) | 10.80 $\pm$ 0.21 (11.0)   |
| 3 (0.25B)                   | 18.19 $\pm$ 0.81 (19.38) | 26.00 $\pm$ 0.89 (27.09) | 11.26 $\pm$ 0.36 (11.7)   |

Table 6: Effect of the gradient-entry threshold  $\lambda_g$  on AlpacaEval 2. Entries are mean  $\pm$  standard deviation over five runs, with the best run in parentheses.

| $\lambda_g$                            | 0                        | $4 \times 10^{-5}$       | $1.8 \times 10^{-4}$     | $2.2 \times 10^{-4}$     | $2.5 \times 10^{-4}$     |
|----------------------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| Percentage of gradient entries updated | 100%                     | 63%                      | 6%                       | 1%                       | 0.15%                    |
| AlpacaEval 2-WR %                      | 15.72 $\pm$ 0.77 (16.34) | 16.02 $\pm$ 0.69 (16.69) | 19.02 $\pm$ 0.62 (20.15) | 19.21 $\pm$ 0.58 (20.25) | 16.10 $\pm$ 0.11 (16.21) |
| AlpacaEval 2-LC %                      | 23.42 $\pm$ 1.03 (24.28) | 24.01 $\pm$ 0.91 (25.10) | 26.06 $\pm$ 0.81 (27.27) | 25.91 $\pm$ 0.95 (27.14) | 23.82 $\pm$ 0.23 (24.00) |

Table 7: Effect of increasing the number of noisy preference pairs used by ComPO. Entries are mean  $\pm$  standard deviation, with the best run in parentheses.

| Number of noisy pairs | AlpacaEval 2-WR %        | AlpacaEval 2-LC %        | Arena-Hard (GPT-4.1)-WR % |
|-----------------------|--------------------------|--------------------------|---------------------------|
| 100                   | 19.21 $\pm$ 0.58 (20.25) | 25.91 $\pm$ 0.95 (27.14) | 11.02 $\pm$ 0.13 (11.2)   |
| 300                   | 20.07 $\pm$ 0.99 (21.35) | 26.28 $\pm$ 0.81 (27.59) | 11.76 $\pm$ 0.30 (12.1)   |

Table 8: Applying ComPO directly to DPO checkpoints without training DPO only on the clean subset. AE, AH, and MT denote AlpacaEval 2, Arena-Hard, and MT-Bench, respectively.

| Method              | AE LC (%) | AE WR (%) | AH (GPT-4.1) WR (%) | MT Turn 1 | MT Turn 2 | MT Avg |
|---------------------|-----------|-----------|---------------------|-----------|-----------|--------|
| DPO                 | 24.14     | 16.71     | 10.40               | 6.28      | 5.42      | 5.86   |
| DPO + ComPO         | 27.03     | 20.85     | 11.40               | 7.80      | 7.61      | 7.71   |
| DPO (clean)         | 23.89     | 16.15     | 10.50               | 6.11      | 5.34      | 5.73   |
| DPO (clean) + ComPO | 27.14     | 20.25     | 11.20               | 7.82      | 7.59      | 7.71   |

**Efficiency and compatibility.** Full fine-tuning and LoRA-based fine-tuning (Hu et al., 2022) are common post-training choices. ComPO instead uses a lightweight update that changes only selected entries in the output layer. Figure 1 (left) shows that the chosen  $\lambda_g$  retains about 1% of the output-layer entries for Mistral-7B and Llama-3-8B. For Mistral-7B, the plotted 0.13B output-layer size and 1.18% retention rate correspond to roughly 1.5 million updated parameters, or about 0.02% of the full 7B model. Except in the multi-layer ablation, parameters outside the output layer remain frozen.

The comparison-based update avoids full-model backpropagation and accumulate signed perturbations without storing all perturbation vectors. Figure 1 (middle) reports a peak of approximately 23 GB per A40 GPU for Llama-3-8B ComPO; the corresponding reported peaks for DPO and SimPO are 77 GB and 69 GB on H100 GPUs. Because these measurements use different hardware, they describe practical resource requirements rather than a controlled head-to-head comparison. ComPO also parallelizes naturally. For 600 perturbations on 30 A40 GPUs, each worker processes 20 perturbations, and the master aggregates the oracle outputs and perturbation signals to form the gradient estimate (Algorithm 2). Figure 1 (right) shows that runtime increases approximately linearly with the perturbed parameter dimension across the three tested models. Except for the multi-layer ablation, perturbations are restricted to the complete `lm_head` layer.

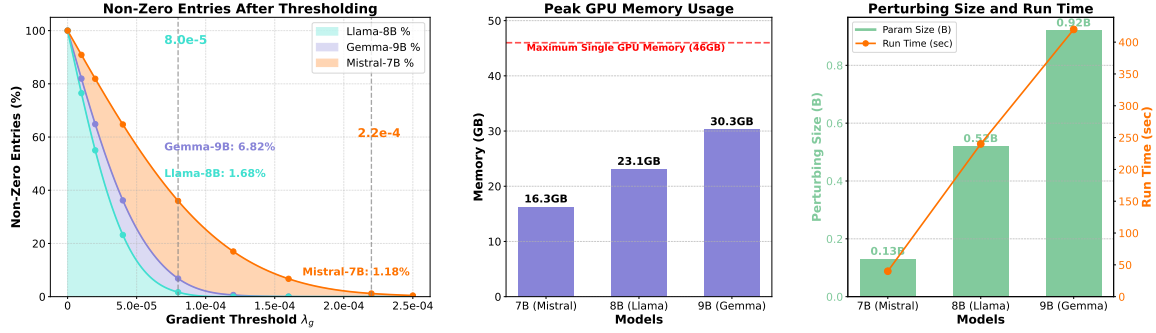


Figure 1: (Left) Percentage of nonzero entries in the final gradient as the gradient-entry threshold  $\lambda_g$  varies. (Middle) Peak GPU memory used by ComPO for the three model families. (Right) Perturbed output-layer size and wall-clock time for completing 600 perturbations on 30 NVIDIA A40 GPUs.

Table 9: Mean  $\pm$  standard deviation of the number of negative oracle outputs for the first ten noisy pairs across eight consecutive runs.

| Pair 1             | Pair 2             | Pair 3             | Pair 4             | Pair 5             |
|--------------------|--------------------|--------------------|--------------------|--------------------|
| 394.25 $\pm$ 28.30 | 364.50 $\pm$ 14.21 | 369.00 $\pm$ 20.39 | 447.00 $\pm$ 19.87 | 591.00 $\pm$ 13.46 |
| Pair 6             | Pair 7             | Pair 8             | Pair 9             | Pair 10            |
| 282.00 $\pm$ 14.98 | 459.25 $\pm$ 10.66 | 242.13 $\pm$ 15.29 | 311.13 $\pm$ 15.87 | 348.75 $\pm$ 18.59 |

ComPO can also be applied directly to an existing checkpoint without first training the underlying DPO model only on clean pairs. In Table 8, we start from DPO checkpoints trained on the full preference dataset and then apply ComPO with  $m = 3300$ . The resulting gains are comparable to those obtained from  $\text{DPO}_{\text{clean}} + \text{ComPO}$ . This supports a practical workflow in which a user starts from a publicly available aligned model and refines it with task-specific, potentially noisy preference data using sparse output-layer updates and modest GPU memory.

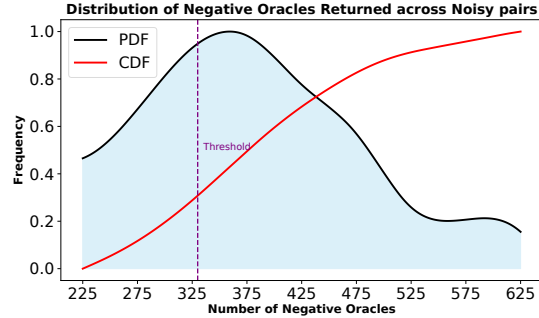


Figure 2: Empirical and cumulative distributions of the number of negative oracle outputs across noisy pairs. The dashed line marks the threshold used for Mistral-7B-Base.

**Successful perturbations and clipping threshold  $\lambda$ .** In addition to the entry-level threshold  $\lambda_g$ , ComPO uses the clipping threshold  $\lambda > 0$  to discard an update when too few perturbations return successful comparison-oracle signals. Figure 2 shows the empirical distribution of the number of negative oracle outputs  $k = |\{i : y_i = -1\}|$  across noisy pairs for Mistral-7B-Base. The threshold removes the low-count tail by skipping updates with a small fraction of favorable perturbations. Table 9 further shows that this count remains in a similar range for a fixed pair across eight independent runs. Together, these results indicate that the amount of usable oracle feedback is reproducible and that clipping avoids poorly supported updates.

Table 10: Evaluation on the GPT-4.1 configurations of AlpacaEval 2 and Arena-Hard. LC and WR denote length-controlled and raw win rates. PA denotes the pre-alignment supervised or instruction-fine-tuned checkpoint. “+RKL” uses the length-normalized damping rule in Algorithm 4 and the “+resampling” adds replay to that same online variant.

| Method             | Qwen3-4B-Base |        |            | Llama-3.2-3B-Instruct |        |            | Gemma-3-4B-it |        |            |
|--------------------|---------------|--------|------------|-----------------------|--------|------------|---------------|--------|------------|
|                    | AlpacaEval 2  |        | Arena-Hard | AlpacaEval 2          |        | Arena-Hard | AlpacaEval 2  |        | Arena-Hard |
|                    | LC (%)        | WR (%) | WR (%)     | LC (%)                | WR (%) | WR (%)     | LC (%)        | WR (%) | WR (%)     |
| PA                 | 12.70         | 13.12  | 16.2       | 11.16                 | 11.83  | 9.8        | 34.54         | 56.20  | 54.8       |
| DPO                | 15.28         | 15.54  | 29.3       | 11.72                 | 12.08  | 11.6       | 38.30         | 57.87  | 56.9       |
| DPO+ComPO          | 16.20         | 16.27  | 30.8       | 12.35                 | 12.50  | 11.9       | 40.00         | 58.57  | 57.7       |
| DPO+ComPO (online) |               |        |            |                       |        |            |               |        |            |
| + RKL              | 17.43         | 17.74  | 31.4       | 12.70                 | 13.23  | 12.4       | 42.07         | 60.40  | 63.3       |
| + resampling       | 18.57         | 17.95  | 32.6       | 13.05                 | 13.85  | 12.8       | 42.55         | 60.93  | 63.7       |

### 4.3 Online training

We evaluate online ComPO in Algorithm 4. It keeps the offline comparison direction and uses unlabeled samples to compute the length-normalized statistic in Eq. (14). This statistic adjusts the step size through the soft-damping rule in Eq. (15). The implementation is a heuristic approximation to the basic scheme in Algorithm 3. Indeed, it does not evaluate the proposed next policy or enforce the hard sequence-level reverse-KL constraint analyzed in Theorem 3.4. For the replay buffer, we set the window length to  $n = 50$ .

For Qwen3-4B-Base, we use  $r = 0.0008$ ,  $m = 1800$ , and  $\lambda_g = 0.000085$ . For Gemma-3-4B-it, we use  $r = 0.00045$ ,  $m = 1800$ , and  $\lambda_g = 0.000075$ . We evaluate Qwen3-4B-Base<sup>6</sup>, Llama-3.2-3B-Instruct<sup>7</sup>, and Gemma-3-4B-it<sup>8</sup> using the GPT-4.1 configurations of AlpacaEval 2 and Arena-Hard. Unless stated otherwise, the remaining training settings follow the offline protocol in Section 4.1.

In Table 10, we compare offline ComPO, ComPO with online damping, and ComPO with both damping and replay. Relative to offline ComPO, damping improves AlpacaEval 2 LC, AlpacaEval 2 WR, and Arena-Hard WR by 1.23, 1.47, and 0.6 percentage points for Qwen3-4B-Base; 0.35, 0.73, and 0.5 points for Llama-3.2-3B-Instruct; and 2.07, 1.83, and 5.6 points for Gemma-3-4B-it. Adding replay improves all three reported metrics for each model. These comparisons support the empirical benefit of the combined procedure in the tested configurations, without identifying a separate variance-reduction mechanism.

## 5 Conclusion

We propose a new zeroth-order preference alignment method based on comparison oracles and show that it can improve large language models (LLMs) using noisy preference pairs for which the reference policy assigns similar likelihoods to preferred and dispreferred responses. The key idea is to use such pairs as comparison signals rather than directly optimizing a preference loss on them. Experimental results on multiple models and benchmarks show that ComPO improves existing direct alignment methods, with pair-level diagnostics pro-

6. <https://huggingface.co/Qwen/Qwen3-4B-Base>

7. <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>

8. <https://huggingface.co/google/gemma-3-4b-it>

viding evidence consistent with mitigating likelihood displacement. These results highlight the importance of designing specialized methods for preference pairs with small likelihood margins, complementing the recent findings of Razin et al. (2025).

The extension in this journal version is online ComPO, where offline noisy preference pairs continue to determine the comparison direction, and unlabeled generations from the current policy provide reverse-KL regularization. We establish feasibility and a coverage-based performance bound for the basic constrained scheme and evaluate damping and replay in the practical implementation. Future directions include extending our approach to other settings (Yuan et al., 2024; Xu et al., 2024b; Tajwar et al., 2024; Guo et al., 2024; Chen and Chen, 2026) and applying it to other tasks, including reasoning (Pang et al., 2024; Chen et al., 2025c) and diffusion model alignment (Wallace et al., 2024).

## Acknowledgement

We sincerely appreciate Buzz High Performance Computing (<https://www.buzzhpc.ai>, [info@buzzhpc.ai](mailto:info@buzzhpc.ai)) for providing computational resources and support for this work. Tianyi Lin gratefully acknowledges financial support through a start-up grant and an early career scholarship support grant at Columbia University.

## References

- J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Al-tenschmidt, S. Altman, S. Anadkat, et al. GPT-4 technical report. *ArXiv Preprint: 2303.08774*, 2023.
- A. Agarwal, O. Dekel, and L. Xiao. Optimal algorithms for online convex optimization with multi-point bandit feedback. In *COLT*, pages 28–40, 2010.
- R. Akrou, M. Schoenauer, and M. Sebag. Preference-based policy learning. In *ECML PKDD*, pages 12–27, 2011.
- A. Amini, T. Vieira, and R. Cotterell. Direct preference optimization with an offset. In *ACL*, pages 9954–9972, 2024.
- R. Astudillo and P. Frazier. Multi-attribute Bayesian optimization with interactive preference learning. In *AISTATS*, pages 4496–4507, 2020.
- M. G. Azar, Z. Guo, B. Piot, R. Munos, M. Rowland, M. Valko, and D. Calandriello. A general theoretical paradigm to understand learning from human preferences. In *AISTATS*, pages 4447–4455, 2024.
- Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *ArXiv Preprint: 2204.05862*, 2022.
- P. T. Boufounos and R. G. Baraniuk. 1-bit compressive sensing. In *CISS*, pages 16–21. IEEE, 2008.
- T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *NeurIPS*, pages 1877–1901, 2020.
- S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, et al. Sparks of artificial general intelligence: Early experiments with GPT-4. *ArXiv Preprint: 2303.12712*, 2023.
- R. Busa-Fekete, B. Szörényi, P. Weng, W. Cheng, and E. Hüllermeier. Preference-based reinforcement learning: Evolutionary direct policy search using a preference-based racing algorithm. *Machine learning*, 97:327–351, 2014.
- H. Cai, D. McKenzie, W. Yin, and Z. Zhang. A one-bit, comparison-based gradient estimator. *Applied and Computational Harmonic Analysis*, 60:242–266, 2022a.
- H. Cai, D. McKenzie, W. Yin, and Z. Zhang. Zeroth-order regularized optimization (ZORO): Approximately sparse gradients and adaptive sampling. *SIAM Journal on Optimization*, 32(2):687–714, 2022b.
- S. Casper, X. Davies, C. Shi, T. K. Gilbert, J. Scheurer, J. Rando, R. Freedman, T. Korbak, D. Lindner, P. Freire, T. T. Wang, S. Marks, C-R. Ségerie, M. Carroll, A. Peng, P. J. K.

- Christoffersen, M. Damani, S. Slocum, U. Anwar, A. Siththaranjan, M. Nadeau, E. J. Michaud, J. Pfau, D. Krashennnikov, X. Chen, L. Langosco, P. Hase, E. Biyik, A. D. Dragan, D. Krueger, D. Sadigh, and D. Hadfield-Menell. Open problems and fundamental limitations of reinforcement learning from human feedback. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=bx24KpJ4Eb>.
- H. Chen, G. He, L. Yuan, G. Cui, H. Su, and J. Zhu. Noise contrastive alignment of language models with explicit rewards. In *NeurIPS*, pages 117784–117812, 2024.
- H. Chen, H. Zhao, H. Lam, D. Yao, and W. Tang. MallowsPO: Fine-tune your LLM with preference dispersions. In *ICLR*, 2025a. URL <https://openreview.net/forum?id=d8cnezVcaW>.
- P. Chen and X. Chen. Two-fidelity best-action identification for stochastic minimax tree. *ArXiv Preprint: 2606.01708*, 2026.
- P. Chen, X. Chen, W. Yin, and T. Lin. ComPO: Preference alignment via comparison oracles. In *NeurIPS*, pages 121962–121995, 2025b.
- P. Chen, X. Li, Z. Li, X. Chen, and T. Lin. Stepwise guided policy optimization: Coloring your incorrect reasoning in GRPO. *Transactions on Machine Learning Research (TMLR)*, 2025c. ISSN 2835-8856. URL <https://openreview.net/forum?id=ALnVAqtshR>.
- P. Chen, X. Li, X. Chen, and T. Lin. Reward-free alignment for conflicting objectives. In *ICML*, 2026a. URL <https://openreview.net/forum?id=vSzRJyg6k0>.
- P. Chen, X. Li, Z. Li, W. Yin, X. Chen, and T. Lin. Exploration vs exploitation: Rethinking RLVR through clipping, entropy, and spurious reward. In *ICLR*, 2026b. URL <https://openreview.net/forum?id=sE8DCSJtZd>.
- X. Chen, S. Liu, K. Xu, X. Li, X. Lin, M. Hong, and D. Cox. ZO-AdaMM: zeroth-order adaptive momentum method for black-box optimization. In *NeurIPS*, pages 7204–7215, 2019.
- M. Cheng, S. Singh, P. H. Chen, P-Y. Chen, S. Liu, and C-J. Hsieh. Sign-OPT: A query-efficient hard-label adversarial attack. In *ICLR*, 2020. URL <https://openreview.net/forum?id=Sk1TQCNTvS>.
- K. Choromanski, A. Pacchiano, J. Parker-Holder, Y. Tang, and V. Sindhvani. From complexity to simplicity: Adaptive ES-Active subspaces for blackbox optimization. In *NeurIPS*, pages 10299–10309, 2019.
- A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *NeurIPS*, pages 4302–4310, 2017.



- E. Conti, V. Madhavan, F. P. Such, J. Lehman, K. O. Stanley, and J. Clune. Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents. In *NeurIPS*, pages 5032–5043, 2018.
- G. Cui, L. Yuan, N. Ding, G. Yao, B. He, W. Zhu, Y. Ni, G. Xie, R. Xie, Y. Lin, Z. Liu, and M. Sun. Ultrafeedback: Boosting language models with scaled AI feedback. In *ICML*, pages 9722–9744, 2024.
- Y-X. Ding and Z-H. Zhou. Preference based adaptation for learning objectives. In *NeurIPS*, pages 7839–7848, 2018.
- H. Dong, W. Xiong, D. Goyal, Y. Zhang, W. Chow, R. Pan, S. Diao, J. Zhang, K. Shum, and T. Zhang. RAFT: Reward ranked fine-tuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=m7p507zblY>.
- H. Dong, W. Xiong, B. Pang, H. Wang, H. Zhao, Y. Zhou, N. Jiang, D. Sahoo, C. Xiong, and T. Zhang. RLHF workflow: From reward modeling to online RLHF. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=a13aYUU9eU>.
- Y. Dubois, X. Li, R. Taori, T. Zhang, I. Gulrajani, J. Ba, C. Guestrin, P. Liang, and T. B. Hashimoto. AlpacaFarm: A simulation framework for methods that learn from human feedback. In *NeurIPS*, pages 30039–30069, 2023.
- Y. Dubois, P. Liang, and T. Hashimoto. Length-controlled AlpacaEval: A simple debiasing of automatic evaluators. In *COLM*, 2024. URL <https://openreview.net/forum?id=CybBmzWBX0>.
- J. C. Duchi, M. I. Jordan, M. J. Wainwright, and A. Wibisono. Optimal rates for zero-order convex optimization: The power of two function evaluations. *IEEE Transactions on Information Theory*, 61(5):2788–2806, 2015.
- K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela. Model alignment as prospect theoretic optimization. In *ICML*, pages 12634–12651, 2024.
- A. D. Flaxman, A. T. Kalai, and H. B. McMahan. Online convex optimization in the bandit setting: Gradient descent without a gradient. In *SODA*, pages 385–394, 2005.
- L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization. In *ICML*, pages 10835–10866, 2023.
- S. Ghadimi and G. Lan. Stochastic first-and zeroth-order methods for nonconvex stochastic programming. *SIAM Journal on Optimization*, 23(4):2341–2368, 2013.
- D. Golovin, J. Karro, G. Kochanski, C. Lee, X. Song, and Q. Zhang. Gradientless descent: High-dimensional zeroth-order optimization. In *ICLR*, 2020. URL <https://openreview.net/forum?id=Skep6TVYDB>.
- S. Guo, B. Zhang, T. Liu, T. Liu, M. Khalman, F. Llinares, A. Rame, T. Mesnard, Y. Zhao, B. Piot, et al. Direct language model alignment from online AI feedback. *arXiv preprint arXiv:2402.04792*, 2024.

- J. Hong, N. Lee, and J. Thorne. ORPO: Monolithic preference optimization without reference model. In *EMNLP*, pages 11170–11189, 2024.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- F. Huang, S. Gao, J. Pei, and H. Huang. Accelerated zeroth-order and first-order momentum methods from mini to minimax optimization. *Journal of Machine Learning Research*, 23(36):1–70, 2022.
- K. G. Jamieson, R. Nowak, and B. Recht. Query complexity of derivative-free optimization. In *NeurIPS*, pages 2672–2680, 2012.
- K. Ji, Z. Wang, Y. Zhou, and Y. Liang. Improved zeroth-order variance reduced algorithms and analysis for nonconvex optimization. In *ICML*, pages 3100–3109, 2019.
- S. Kabir, D. N. Udo-Imeh, B. Kou, and T. Zhang. Is stack overflow obsolete? an empirical study of the characteristics of ChatGPT answers to stack overflow questions. In *CHI*, pages 1–17, 2024.
- K. Kim, A. Seo, H. Liu, J. Shin, and K. Lee. Margin matching preference optimization: Enhanced model alignment with granular feedback. In *EMNLP*, pages 13554–13570, 2024.
- G. Kornowski and O. Shamir. An algorithm with optimal dimension-dependence for zero-order nonsmooth nonconvex stochastic optimization. *Journal of Machine Learning Research*, 25(122):1–14, 2024.
- W. Kumagai. Regret analysis for continuous dueling bandit. In *NeurIPS*, pages 1488–1497, 2017.
- T. Li, W-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez, and I. Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *ArXiv Preprint: 2406.11939*, 2024.
- X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, and T. B. Hashimoto. AlpacaEval: An automatic evaluator of instruction-following models. [https://github.com/tatsu-lab/alpaca\\_eval](https://github.com/tatsu-lab/alpaca_eval), 5 2023.
- X. Lian, H. Zhang, C-J. Hsieh, Y. Huang, and J. Liu. A comprehensive linear speedup analysis for asynchronous stochastic parallel optimization from zeroth-order to first-order. In *NeurIPS*, pages 3062–3070, 2016.
- T. Lin, Z. Zheng, and M. I. Jordan. Gradient-free methods for deterministic and stochastic nonsmooth nonconvex optimization. In *NeurIPS*, pages 26160–26175, 2022a.
- Z. J. Lin, R. Astudillo, P. Frazier, and E. Bakshy. Preference exploration for efficient Bayesian optimization with multiple outcomes. In *AISTATS*, pages 4235–4258, 2022b.
- S. Liu, B. Kailkhura, P-Y. Chen, P. Ting, S. Chang, and L. Amini. Zeroth-order stochastic variance reduction for nonconvex optimization. In *NeurIPS*, pages 3731–3741, 2018.

- T. Liu, Y. Zhao, R. Joshi, M. Khalman, M. Saleh, P. J. Liu, and J. Liu. Statistical rejection sampling improves preference optimization. In *ICLR*, 2024a. URL <https://openreview.net/forum?id=xbjSwwrQ0e>.
- T. Liu, Z. Qin, J. Wu, J. Shen, M. Khalman, R. Joshi, Y. Zhao, M. Saleh, S. Baumgartner, J. Liu, et al. LiPO: Listwise preference optimization through learning-to-rank. In *NAACL*, page To appear, 2025.
- Z. Liu, M. Lu, S. Zhang, B. Liu, H. Guo, Y. Yang, J. Blanchet, and Z. Wang. Provably mitigating overoptimization in RLHF: Your SFT loss is implicitly an adversarial regularizer. In *NeurIPS*, pages 138663–138697, 2024b.
- S. Malladi, T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora. Fine-tuning language models with just forward passes. In *NeurIPS*, pages 53038–53075, 2023.
- K. Matsui, W. Kumagai, and T. Kanamori. Parallel distributed block coordinate descent methods based on pairwise comparison oracle. *Journal of Global Optimization*, 69:1–21, 2017.
- Y. Meng, M. Xia, and D. Chen. SimPO: Simple preference optimization with a reference-free reward. In *NeurIPS*, pages 124198–124235, 2024.
- Y. Nesterov and V. Spokoiny. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017.
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, pages 27730–27744, 2022.
- A. Pal, D. Karkhanis, S. Dooley, M. Roberts, S. Naidu, and C. White. Smaug: Fixing failure modes of preference optimisation with DPO-positive. *ArXiv Preprint: 2402.13228*, 2024.
- R. Y. Pang, W. Yuan, H. He, K. Cho, S. Sukhbaatar, and J. Weston. Iterative reasoning preference optimization. In *NeurIPS*, pages 116617–116637, 2024.
- R. Park, R. Rafailov, S. Ermon, and C. Finn. Disentangling length from quality in direct preference optimization. In *ACL*, pages 4998–5017, 2024.
- Y. Plan and R. Vershynin. Robust 1-bit compressed sensing and sparse logistic regression: A convex programming approach. *IEEE Transactions on Information Theory*, 59(1): 482–494, 2012.
- R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, pages 53728–53741, 2023.
- R. Rafailov, Y. Chittapu, R. Park, H. Sikchi, J. Hejna, W. B. Knox, C. Finn, and S. Niekum. Scaling laws for reward model overoptimization in direct alignment algorithms. In *NeurIPS*, pages 126207–126242, 2024a.

- R. Rafailov, J. Hejna, R. Park, and C. Finn. From  $\$r\$$  to  $\$q^*\$$ : Your language model is secretly a Q-function. In *COLM*, 2024b. URL <https://openreview.net/forum?id=kEVcNxtqXk>.
- N. Razin, S. Malladi, A. Bhaskar, D. Chen, S. Arora, and B. Hanin. Unintentional unalignment: Likelihood displacement in direct preference optimization. In *ICLR*, 2025. URL <https://openreview.net/forum?id=uaMSBJDnRv>.
- Y. Ren and D. J. Sutherland. Learning dynamics of LLM finetuning. In *ICLR*, 2025. URL <https://openreview.net/forum?id=tPNH0oZF19>.
- T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *ArXiv Preprint: 1703.03864*, 2017.
- O. Shamir. An optimal algorithm for bandit and zero-order convex optimization with two-point feedback. *Journal of Machine Learning Research*, 18(1):1703–1713, 2017.
- R. Shi, R. Zhou, and S. S. Du. The crucial role of samplers in online direct preference optimization. In *ICLR*, 2025. URL <https://openreview.net/forum?id=F6z3utfcYw>.
- P. Singhal, T. Goyal, J. Xu, and G. Durrett. A long way to go: Investigating length correlations in RLHF. In *COLM*, 2024. URL <https://openreview.net/forum?id=G8La01P0xv>.
- F. Song, B. Yu, M. Li, H. Yu, F. Huang, Y. Li, and H. Wang. Preference ranking optimization for human alignment. In *AAAI*, pages 18990–18998, 2024a.
- Y. Song, G. Swamy, A. Singh, J. Bagnell, and W. Sun. The importance of online data: Understanding preference fine-tuning via coverage. In *NeurIPS*, pages 12243–12270, 2024b.
- N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. In *NeurIPS*, pages 3008–3021, 2020.
- F. Tajwar, A. Singh, A. Sharma, R. Rafailov, J. Schneider, T. Xie, S. Ermon, C. Finn, and A. Kumar. Preference fine-tuning of LLMs should leverage suboptimal, on-policy data. In *ICML*, pages 47441–47474, 2024.
- Y. Tang, Z. Guo, Z. Zheng, D. Calandriello, R. Munos, M. Rowland, P. H. Richemond, M. Valko, B. Pires, and B. Piot. Generalized preference optimization: A unified approach to offline alignment. In *ICML*, pages 47725–47742, 2024a.
- Z. Tang, D. Rybin, and T-H. Chang. Zeroth-order optimization meets human feedback: Provable learning via ranking oracles. In *ICLR*, 2024b. URL <https://openreview.net/forum?id=TVDUVpgu9s>.
- H. Touvron, T. Lavril, G. Izacard, X. Martinet, M-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *ArXiv Preprint: 2302.13971*, 2023.

- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, pages 6000–6010, 2017.
- B. Wallace, M. Dang, R. Rafailov, L. Zhou, A. Lou, S. Purushwalkam, S. Ermon, C. Xiong, S. Joty, and N. Naik. Diffusion model alignment using direct preference optimization. In *CVPR*, pages 8228–8238, 2024.
- Y. Wang, S. Du, S. Balakrishnan, and A. Singh. Stochastic zeroth-order optimization in high dimensions. In *AISTATS*, pages 1356–1365, 2018.
- T. Xiao, Y. Yuan, H. Zhu, M. Li, and V. G. Honavar. Cal-DPO: Calibrated direct preference optimization for language model alignment. In *NeurIPS*, pages 114289–114320, 2024.
- T. Xie, D. J. Foster, A. Krishnamurthy, C. Rosset, A. H. Awadallah, and A. Rakhlin. Exploratory preference optimization: Harnessing implicit  $q^*$ -approximation for sample-efficient RLHF. In *ICLR*, 2025. URL <https://openreview.net/forum?id=QYigQ6gXNw>.
- W. Xiong, H. Dong, C. Ye, Z. Wang, H. Zhong, H. Ji, N. Jiang, and T. Zhang. Iterative preference learning from human feedback: Bridging theory and practice for RLHF under KL-constraint. In *ICML*, pages 54715–54754, 2024.
- H. Xu, A. Sharaf, Y. Chen, W. Tan, L. Shen, B. Van Durme, K. Murray, and Y. J. Kim. Contrastive preference optimization: Pushing the boundaries of LLM performance in machine translation. In *ICML*, pages 55204–55224, 2024a.
- S. Xu, W. Fu, J. Gao, W. Ye, W. Liu, Z. Mei, G. Wang, C. Yu, and Y. Wu. Is DPO superior to PPO for LLM alignment? a comprehensive study. In *ICML*, pages 54983–54998, 2024b.
- H. Yuan, Z. Yuan, C. Tan, W. Wang, S. Huang, and F. Huang. RRHF: Rank responses to align language models with human feedback. In *NeurIPS*, pages 10935–10950, 2023.
- L. Yuan, G. Cui, H. Wang, N. Ding, X. Wang, B. Shan, Z. Liu, J. Deng, H. Chen, R. Xie, Y. Lin, Z. Liu, B. Zhou, H. Peng, Z. Liu, and M. Sun. Advancing LLM reasoning generalists with preference trees. In *ICLR*, 2025. URL <https://openreview.net/forum?id=2ea5TNVR0c>.
- W. Yuan, R. Y. Pang, K. Cho, X. Li, S. Sukhbaatar, J. Xu, and J. E. Weston. Self-rewarding language models. In *ICML*, pages 57905–57923, 2024.
- Y. Yue and T. Joachims. Interactively optimizing information retrieval systems as a dueling bandits problem. In *ICML*, pages 1201–1208, 2009.
- Q. Zhang and L. Ying. Zeroth-order policy gradient for reinforcement learning from human feedback without reward inference. In *ICLR*, 2025. URL <https://openreview.net/forum?id=cmYScmfu4Q>.
- S. Zhang, Z. Liu, B. Liu, Y. Zhang, Y. Yang, Y. Liu, L. Chen, T. Sun, and Z. Wang. Reward-augmented data enhances direct preference alignment of LLMs. In *ICLR Workshop on Navigating and Addressing Data Problems for Foundation Models*, 2025. URL <https://openreview.net/forum?id=bpSD3IOgyS>.

- Y. Zhang, P. Li, J. Hong, J. Li, Y. Zhang, W. Zheng, P-Y. Chen, J. D. Lee, W. Yin, M. Hong, et al. Revisiting zeroth-order optimization for memory-efficient LLM fine-tuning: a benchmark. In *ICML*, pages 59173–59190, 2024.
- H. Zhao, G. I. Winata, A. Das, S-X. Zhang, D. Yao, W. Tang, and S. Sahu. RainbowPO: A unified framework for combining improvements in preference optimization. In *ICLR*, 2025. URL <https://openreview.net/forum?id=trKee5pIFv>.
- Y. Zhao, R. Joshi, T. Liu, M. Khalman, M. Saleh, and P. J. Liu. SLiC-HF: Sequence likelihood calibration with human feedback. *ArXiv Preprint: 2305.10425*, 2023.
- L. Zheng, W-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, and E. P. Xing. Judging LLM-as-a-Judge with MT-bench and Chatbot Arena. In *NeurIPS*, pages 46595–46623, 2023.
- B. Zhu, M. I. Jordan, and J. Jiao. Principled reinforcement learning with human feedback from pairwise or k-wise comparisons. In *ICML*, pages 43037–43067, 2023.
- D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving. Fine-tuning language models from human preferences. *ArXiv Preprint: 1909.08593*, 2019.

## Appendix A. Further Related Work

We make additional comments on other topics, including preference learning methods, the analysis of preference learning methods, zeroth-order optimization methods, likelihood displacement, and learning from noisy preference data. For an overview of preference learning methods and open problems in RLHF, we refer to the recent survey (Casper et al., 2023).

**More discussion on preference learning methods.** The lack of explicit reward models in DPO (Rafailov et al., 2023) is known to make its performance depend strongly on the size and quality of offline preference pairs. To address this limitation, subsequent works proposed to augment preference data using a trained SFT policy (Zhao et al., 2023) or a refined SFT policy with rejection sampling (Liu et al., 2024a). The DPO loss was also extended to a token-level MDP (Rafailov et al., 2024b), where the transition is deterministic, i.e., the next state is determined once the current state and action are chosen, which naturally covers the fine-tuning of autoregressive LLMs. Azar et al. (2024) further generalized DPO to a wider class of RL problems without explicitly introducing a reward function. Instead of maximizing a reward in a KL-constrained problem, they proposed to optimize a general non-decreasing function of the ground-truth population-level preference probability. There are also several other DPO variants (Ethayarajh et al., 2024; Park et al., 2024; Xu et al., 2024a; Meng et al., 2024; Chen et al., 2025a; Zhao et al., 2025). For example, Ethayarajh et al. (2024) aligned the policy with preferences using a prospect-theoretic loss, Tang et al. (2024a) optimized a general loss instead of the log-likelihood loss, and Meng et al. (2024) aligned the reward function in the preference optimization objective with the generation metric. Dong et al. (2024) and Xiong et al. (2024) proposed to generate human feedback in an online fashion to mitigate distribution shift and over-optimization. There has also been an attempt to understand the theoretical performance of DPO (Azar et al., 2024), although this analysis mainly focuses on the population-level objective rather than finite-sample policy-optimality or sample-complexity guarantees. Chen et al. (2026a) also extends DPO-style direct alignment to multiple-objective setup via a novel conflict-averse formulation.

**Analysis of preference learning methods.** In this context, Zhu et al. (2023) formulated RLHF as a contextual bandit problem and proved the convergence of the maximum likelihood estimator. Xiong et al. (2024) showed the benefits of KL regularization for the sample complexity of online exploration in DPO. Xie et al. (2025) studied online exploration using KL-regularized Markov decision processes and proved a sample-complexity guarantee for an exploration bonus. Liu et al. (2024b) investigated the issue of over-optimization and proved finite-sample guarantees. Song et al. (2024b) conducted a rigorous analysis through the lens of dataset coverage to differentiate offline DPO and online RLHF. Recently, several works have reported faster convergence rates for online reward maximization in RL by exploiting the structure induced by KL regularization. For example, Shi et al. (2025) studied the tabular softmax parametrization setting and established quadratic convergence results.

**Zeroth-order optimization methods.** The idea of zeroth-order optimization is to approximate a gradient using either a one-point estimator (Flaxman et al., 2005) or a two-point estimator (Agarwal et al., 2010; Ghadimi and Lan, 2013; Duchi et al., 2015; Shamir, 2017; Nesterov and Spokoiny, 2017), where the latter approach often achieves better finite-time convergence guarantees. Despite the rapid development of two-point-based gradient-free



methods, much of the work focuses on convex optimization (Duchi et al., 2015; Shamir, 2017; Wang et al., 2018) and smooth nonconvex optimization (Nesterov and Spokoiny, 2017; Ghadimi and Lan, 2013; Lian et al., 2016; Liu et al., 2018; Chen et al., 2019; Ji et al., 2019; Huang et al., 2022). Convergence guarantees have been obtained in both nonsmooth convex settings (Duchi et al., 2015; Shamir, 2017) and smooth nonconvex settings (Ghadimi and Lan, 2013; Nesterov and Spokoiny, 2017). Additional regularity conditions, e.g., a finite-sum structure, allow variance-reduction techniques to be used (Liu et al., 2018; Chen et al., 2019; Ji et al., 2019), and sharp convergence guarantees are obtained in Huang et al. (2022). Very recently, zeroth-order optimization methods have been developed for nonsmooth nonconvex optimization with solid theoretical guarantees (Lin et al., 2022a; Kornowski and Shamir, 2024). In another direction, zeroth-order optimization methods were extended to the RL setting and have achieved empirical success as scalable alternatives to classic methods such as Q-learning and policy gradient methods (Salimans et al., 2017; Conti et al., 2018). This strategy has also been applied in preference-based RL (Akrouer et al., 2011; Busa-Fekete et al., 2014) and adopted for LLM fine-tuning (Malladi et al., 2023; Zhang et al., 2024). In these settings, the loss function can be explicitly estimated or calculated and thus can be queried to construct the gradient estimator. By contrast, our method and the methods of Tang et al. (2024b) and Zhang and Ying (2025) are developed based on comparison oracles or ranking oracles, where even noisy estimates of loss-function values are not accessible.

**Likelihood displacement.** We provide a brief overview of proposed explanations for likelihood displacement. Indeed, several works claimed that samples with similar preferred and dispreferred responses are responsible for likelihood displacement (Pal et al., 2024; Tajwar et al., 2024; Razin et al., 2025), although the similarities were measured using different metrics. Other proposed reasons include effects of the initial SFT model (Rafailov et al., 2024b), the presence of multiple training samples and limited model capacity (Tajwar et al., 2024), and the squeezing effect (Ren and Sutherland, 2025). Recently, Razin et al. (2025) conducted a thorough investigation to understand the causes of likelihood displacement, and their results suggest that samples with similar preferred and dispreferred responses might contribute more than others. Regarding the implications of likelihood displacement, previous works found that DPO tends to degrade performance on math and reasoning (Pal et al., 2024; Pang et al., 2024; Meng et al., 2024; Yuan et al., 2025). Indeed, only a few responses are correct, and likelihood displacement can have adverse effects on correct alignment.

**Learning from noisy preference data.** ComPO addresses low-margin preference pairs selected by Eq. (10). This setting is related to, but distinct from, learning with corrupted preference labels (Amini et al., 2024; Xiao et al., 2024). From this perspective, ComPO is not intended as a direct replacement for existing methods, but rather as a complementary and modular component that enhances their robustness. Moreover, learning from corrupted preference data has been studied in prior works, including those leveraging reward scores through conditional DPO (Kim et al., 2024; Zhang et al., 2025). Conditional DPO modifies the DPO objective by conditioning on reward scores and solves the resulting problem via gradient-based methods, and it can be combined with ComPO in a way similar to SimPO+ComPO as in our work. Apart from noisy labels, Chen et al. (2026b) also theoretically analyze the impact false-positive and false-negative labels in online LLM RL, which is complementary to the mis-labeled preference pairs.

## Appendix B. Missing Proofs

We present several technical lemmas and use them to prove Theorem 3.2 and Theorem 3.4.

### B.1 Technical lemmas

For the offline analysis, we fix a nonempty set  $S \subseteq D$  and impose the smoothness, gradient sparsity, and oracle compatibility (see Theorem 3.2) throughout this subsection. We use  $\text{sign}(0) = +1$ . At any point with  $\nabla f(\theta) \neq 0$ , we write

$$y_i = C_\pi^S(\theta, \theta + r\mathbf{z}_i), \quad \bar{\mathbf{g}} = \frac{\nabla f(\theta)}{\|\nabla f(\theta)\|}, \quad \bar{y}_i = \text{sign}(\mathbf{z}_i^\top \bar{\mathbf{g}}).$$

Thus, the oracle compatibility guarantees  $y_i = \text{sign}(f(\theta + r\mathbf{z}_i) - f(\theta))$ . The next proposition adapts the one-bit estimation framework (Plan and Vershynin, 2012; Cai et al., 2022a) to errors that might depend on the perturbation directions but are localized near directions orthogonal to the target.

**Proposition B.1** *Let  $1 \leq s \leq d$  and let  $\bar{\mathbf{g}} \in \mathbb{R}^d$  satisfy  $\|\bar{\mathbf{g}}\|_1 \leq \sqrt{s}$  and  $\|\bar{\mathbf{g}}\| = 1$ . Suppose that  $(\mathbf{z}_i, y_i)_{i=1}^m$  are i.i.d., where  $\mathbf{z}_i$  is uniform on the unit sphere in  $\mathbb{R}^d$  and  $y_i \in \{-1, +1\}$ , and  $y_i = \text{sign}(\mathbf{z}_i^\top \bar{\mathbf{g}})$  almost surely whenever  $|\mathbf{z}_i^\top \bar{\mathbf{g}}| > \frac{1}{40\sqrt{d}}$ . Then, we define*

$$\hat{\mathbf{g}} \in \underset{\|\mathbf{g}\|_1 \leq \sqrt{s}, \|\mathbf{g}\| \leq 1}{\text{argmax}} \sum_{i=1}^m y_i \mathbf{z}_i^\top \mathbf{g}.$$

For any  $\delta \in (0, 1)$ , if  $m \geq c_m \left( s \log \left( \frac{2d}{s} \right) + \log \left( \frac{2}{\delta} \right) \right)$  for a sufficiently large constant  $c_m$ , we have

$$\mathbb{P} \left( \|\hat{\mathbf{g}} - \bar{\mathbf{g}}\| \leq \frac{1}{2} \right) \geq 1 - \delta.$$

**Proof** For the case of  $d = 1$ , we have  $s = 1$  and  $\mathbf{z}_i, \bar{\mathbf{g}} \in \{-1, +1\}$ . Since  $|\mathbf{z}_i^\top \bar{\mathbf{g}}| = 1$ , the assumption on the labels implies  $y_i = \mathbf{z}_i^\top \bar{\mathbf{g}}$  almost surely. Thus,  $y_i \mathbf{z}_i = \bar{\mathbf{g}}$  almost surely, and the definition of  $\hat{\mathbf{g}}$  gives  $\hat{\mathbf{g}} = \bar{\mathbf{g}}$ . For the case of  $d \geq 2$ , we define

$$K = \{\mathbf{g} \in \mathbb{R}^d : \|\mathbf{g}\|_1 \leq \sqrt{s}, \|\mathbf{g}\| \leq 1\}, \quad F_m(\mathbf{g}) = \frac{1}{m} \sum_{i=1}^m y_i \mathbf{z}_i^\top \mathbf{g}.$$

Since  $\bar{\mathbf{g}} \in K$  and  $\hat{\mathbf{g}}$  maximizes  $F_m$  over  $K$ , it suffices to show, with probability at least  $1 - \delta$ , that  $F_m(\bar{\mathbf{g}}) > F_m(\mathbf{g})$  for every  $\mathbf{g} \in K$  with  $\|\mathbf{g} - \bar{\mathbf{g}}\| > \frac{1}{2}$ . For simplicity, we let  $(\mathbf{z}, y)$  have the same distribution as  $(\mathbf{z}_i, y_i)$ . The key decomposition is given by

$$\frac{1}{m} \sum_{i=1}^m y_i \mathbf{z}_i = \underbrace{\mathbb{E}[\text{sign}(\mathbf{z}^\top \bar{\mathbf{g}}) \mathbf{z}]}_A + \underbrace{\mathbb{E}[(y - \text{sign}(\mathbf{z}^\top \bar{\mathbf{g}})) \mathbf{z}]}_B + \underbrace{\frac{1}{m} \sum_{i=1}^m y_i \mathbf{z}_i - \mathbb{E}[y \mathbf{z}]}_B.$$

We set  $\kappa = \mathbb{E}[|\mathbf{z}^\top \bar{\mathbf{g}}|]$  and obtain from rotational invariance that  $\mathbb{E}[\text{sign}(\mathbf{z}^\top \bar{\mathbf{g}}) \mathbf{z}] = \kappa \bar{\mathbf{g}}$ . Thus, for every  $\mathbf{g} \in K$ , we have

$$F_m(\bar{\mathbf{g}}) - F_m(\mathbf{g}) = \kappa(1 - \bar{\mathbf{g}}^\top \mathbf{g}) + A^\top(\bar{\mathbf{g}} - \mathbf{g}) + B^\top(\bar{\mathbf{g}} - \mathbf{g}). \quad (16)$$

In what follows, we write  $r = \|\mathbf{g} - \bar{\mathbf{g}}\|$  and prove that  $r > \frac{1}{2}$  implies  $F_m(\bar{\mathbf{g}}) - F_m(\mathbf{g}) > 0$ .

**First Term.** Since  $\|\bar{\mathbf{g}}\| = 1$  and  $\|\mathbf{g}\| \leq 1$ , we have  $1 - \bar{\mathbf{g}}^\top \mathbf{g} = \frac{1}{2}(r^2 + 1 - \|\mathbf{g}\|^2) \geq \frac{r^2}{2}$ . Thus, we have

$$\kappa(1 - \bar{\mathbf{g}}^\top \mathbf{g}) \geq \frac{\kappa r^2}{2}. \quad (17)$$

In addition, we prove a lower bound on  $\kappa$ . Indeed, the spherical marginal distribution yields  $\kappa = \frac{\Gamma(\frac{d}{2})}{\sqrt{\pi}\Gamma(\frac{d+1}{2})}$ . Using the log-convexity of the gamma function, we have

$$\left(\Gamma\left(\frac{d+1}{2}\right)\right)^2 \leq \Gamma\left(\frac{d}{2}\right)\Gamma\left(\frac{d+2}{2}\right) = \frac{d}{2} \left(\Gamma\left(\frac{d}{2}\right)\right)^2.$$

which implies the desired bound  $\kappa \geq \sqrt{\frac{2}{\pi d}}$ .

**Second Term.** The key is to prove  $\|A\| \leq \frac{\kappa}{5}$ . Indeed, we set  $q = \mathbf{z}^\top \bar{\mathbf{g}}$ . By assumption,  $y - \text{sign}(q)$  is 0 almost surely outside  $\{|q| \leq \frac{1}{40\sqrt{d}}\}$  and is bounded by 2. The density of  $q$  is  $h_d(q) = \frac{\Gamma(\frac{d}{2})}{\sqrt{\pi}\Gamma(\frac{d-1}{2})}(1 - q^2)^{\frac{d-3}{2}}$  defined on  $q \in (-1, 1)$ . We claim that this density is bounded by  $\sqrt{d}$  if  $|q| \leq \frac{1}{40\sqrt{d}}$ . Indeed, we have

$$h_2(q) = \frac{1}{\pi\sqrt{1-q^2}} \leq \frac{1}{\pi\sqrt{1-a^2/2}} < \sqrt{2}, \quad h_d(q) \leq h_d(0) \leq \sqrt{\frac{d-1}{2\pi}} \leq \sqrt{d} \text{ for } d \geq 3.$$

This implies

$$\mathbb{P}(|q| \leq \frac{1}{40\sqrt{d}}) \leq 2 \cdot \sqrt{d} \cdot \frac{1}{40\sqrt{d}} = \frac{1}{20}.$$

Since the component of  $\mathbf{z}$  orthogonal to  $\bar{\mathbf{g}}$  is rotationally symmetric and has squared norm  $1 - q^2$  conditioned on  $q$ , we have

$$\mathbb{E}[|v^\top \mathbf{z}| \mid q] \leq |v^\top \bar{\mathbf{g}}||q| + \sqrt{\frac{1-q^2}{d-1}}\|v - (v^\top \bar{\mathbf{g}})\bar{\mathbf{g}}\| \leq |q| + \frac{1}{\sqrt{d-1}}$$

for every unit vector  $v$ . It follows that

$$\begin{aligned} |v^\top A| &\leq 2\mathbb{E}\left[|v^\top \mathbf{z}|\mathbf{1}_{\{|q| \leq \frac{1}{40\sqrt{d}}\}}\right] \leq 2\left(\frac{1}{40\sqrt{d}} + \frac{1}{\sqrt{d-1}}\right)\mathbb{P}\left(|q| \leq \frac{1}{40\sqrt{d}}\right) \\ &\leq 4a\left(\frac{a}{\sqrt{d}} + \frac{1}{\sqrt{d-1}}\right) < \frac{1}{5}\sqrt{\frac{2}{\pi d}} \leq \frac{\kappa}{5}. \end{aligned}$$

Taking the supremum over all unit vectors  $v$  yields the desired result. Thus, we have

$$b^\top (\bar{\mathbf{g}} - \mathbf{g}) \geq -\frac{\kappa r}{5}. \quad (18)$$

**Third Term.** The key is to prove  $\sup_{\mathbf{g} \in K} |C^\top \mathbf{g}| \leq \frac{\kappa}{80}$  with probability at least  $1 - \delta$ . Indeed, for every fixed unit vector  $v$  and integer  $k \geq 1$ , the identity  $|y_i \mathbf{z}_i^\top v| = |\mathbf{z}_i^\top v|$  gives

$$\mathbb{E}[|y_i \mathbf{z}_i^\top v|^{2k}] = \frac{(2k-1)!!}{d(d+2)\cdots(d+2k-2)} \leq \frac{(2k-1)!!}{d^k},$$

which imply that  $y_i \mathbf{z}_i^\top v - \mathbb{E}[y_i \mathbf{z}_i^\top v]$  is sub-Gaussian with scale at most  $\frac{C}{\sqrt{d}}$  even though  $y_i$  may depend on  $\mathbf{z}_i$ . Independence across  $i$  yields  $\mathbb{P}(|C^\top v| > h) \leq 2\exp(-c_0 m d h^2)$  for any  $h > 0$  where  $c_0 > 0$  is a universal constant.

We define  $\Omega = \sup_{\|v\| \leq 1, |\text{supp}(v)| \leq \lceil s \rceil} |C^\top v|$ . For each coordinate support  $J$  of size  $\lceil s \rceil$ , we take a  $\frac{1}{2}$ -net  $\mathcal{N}_J$  of its unit sphere with at most  $5^{\lceil s \rceil}$  points. The net approximation gives  $\Omega \leq 2 \max_{|J|=\lceil s \rceil} \max_{v \in \mathcal{N}_J} |C^\top v|$ . A union bound therefore yields

$$\mathbb{P}(\Omega > 2h) \leq 2 \binom{d}{\lceil s \rceil} 5^{\lceil s \rceil} \exp(-c_0 m d h^2).$$

Since  $s \leq \lceil s \rceil \leq 2s$  and  $\lceil s \rceil \leq d$ , we have

$$\log \left( \binom{d}{\lceil s \rceil} 5^{\lceil s \rceil} \right) \leq c_1 s \log \left( \frac{2d}{s} \right).$$

This implies, with probability at least  $1 - \delta$ , we have

$$\Omega \leq c_2 \sqrt{\frac{s \log(2d/s) + \log(2/\delta)}{md}},$$

where  $c_2$  is a universal constant.

To extend this bound to  $K$ , we fix  $\mathbf{g} \in K$ , arrange its coordinates in decreasing magnitude, and partition them into consecutive blocks  $I_1, I_2, \dots$  of size  $\lceil s \rceil$ , with the last block possibly smaller. For every  $j \geq 2$ , we have  $\|\mathbf{g}_{I_j}\| \leq \frac{\|\mathbf{g}_{I_{j-1}}\|_1}{\sqrt{\lceil s \rceil}}$  which implies

$$\sum_j \|\mathbf{g}_{I_j}\| \leq \|\mathbf{g}\| + \frac{\|\mathbf{g}\|_1}{\sqrt{\lceil s \rceil}} \leq 2.$$

Since each block is supported on at most  $\lceil s \rceil$  coordinates, we have  $|C^\top \mathbf{g}| \leq \Omega(\sum_j \|\mathbf{g}_{I_j}\|) \leq 2S$ . It follows that, on the same event, we have

$$\sup_{\mathbf{g} \in K} |C^\top \mathbf{g}| \leq 2c_2 \sqrt{\frac{s \log(2d/s) + \log(2/\delta)}{md}}.$$

Combining this inequality with  $\kappa \geq \sqrt{\frac{2}{\pi d}}$  and  $m \geq c_m (s \log(\frac{2d}{s}) + \log(\frac{2}{\delta}))$  for a sufficiently large constant  $c_m$  yields the desired result. Since  $\bar{\mathbf{g}}$  and  $\mathbf{g}$  belong to  $K$ , we have

$$C^\top (\bar{\mathbf{g}} - \mathbf{g}) \geq -\frac{\kappa}{40}. \quad (19)$$

**End.** On the event established above, Eq. (17), Eq. (18) and Eq. (19) hold simultaneously for every  $\mathbf{g} \in K$ . Since  $r > \frac{1}{2}$ , we have

$$F_m(\bar{\mathbf{g}}) - F_m(\mathbf{g}) \geq \frac{\kappa r^2}{2} - \frac{\kappa r}{5} - \frac{\kappa}{40} = \frac{\kappa}{40} (2r - 1)(10r + 1) > 0.$$

Thus, every feasible vector farther than  $\frac{1}{2}$  from  $\bar{\mathbf{g}}$  has a strictly smaller empirical objective than  $\bar{\mathbf{g}}$  and cannot be a maximizer. In other word,  $\|\hat{\mathbf{g}} - \bar{\mathbf{g}}\| \leq \frac{1}{2}$  on an event of probability at least  $1 - \delta$ . This completes the proof.  $\blacksquare$

The next two lemmas give the descent inequality used to prove the convergence guarantee.

**Lemma B.2** Suppose that  $\|\nabla f(\theta)\| > \frac{\epsilon}{2}$  and  $r = \frac{\epsilon}{40\ell\sqrt{d}}$ . Then, for  $\{\mathbf{z}_i\}_{i=1}^m$  drawn uniformly from the unit sphere in  $\mathbb{R}^d$ , we have  $y_i = \bar{y}_i$  with  $y_i = C_\pi^S(\theta, \theta + r\mathbf{z}_i)$  and  $\bar{y}_i = \text{sign} \left( \mathbf{z}_i^\top \frac{\nabla f(\theta)}{\|\nabla f(\theta)\|} \right)$  whenever  $\left| \mathbf{z}_i^\top \frac{\nabla f(\theta)}{\|\nabla f(\theta)\|} \right| > \frac{1}{40\sqrt{d}}$ .

**Proof** Since  $f$  is  $\ell$ -smooth and  $\|\mathbf{z}_i\| = 1$ , we have

$$|f(\theta + r\mathbf{z}_i) - f(\theta) - r\mathbf{z}_i^\top \nabla f(\theta)| \leq \frac{\ell r^2}{2}.$$

By the choice of  $r$ , we have  $\frac{\ell r}{2} = \frac{\epsilon}{80\sqrt{d}}$ . Since  $\|\nabla f(\theta)\| > \frac{\epsilon}{2}$  and  $\left| \mathbf{z}_i^\top \frac{\nabla f(\theta)}{\|\nabla f(\theta)\|} \right| > \frac{1}{40\sqrt{d}}$ , we have  $r|\mathbf{z}_i^\top \nabla f(\theta)| > \frac{\ell r^2}{2}$ . Putting these pieces together yields

$$y_i = \text{sign}(f(\theta + r\mathbf{z}_i) - f(\theta)) = \text{sign}(\mathbf{z}_i^\top \nabla f(\theta)) = \bar{y}_i.$$

This completes the proof. ■

**Lemma B.3** *Under the stated assumptions, we let  $T \geq 1$ ,  $\eta > 0$ ,  $\epsilon > 0$ , and  $\Lambda \in (0, 1)$ , and set  $r = \frac{\epsilon}{40\ell\sqrt{d}}$ . Suppose that Algorithm 1 uses independent perturbations at every iteration and solves Eq. (9) exactly, and  $m \geq c_0 \left( s \log \left( \frac{2d}{s} \right) + \log \left( \frac{2T}{\Lambda} \right) \right)$  for a sufficiently large constant  $c_0 > 0$ . Then, with probability at least  $1 - \Lambda$ , if  $\min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| > \frac{\epsilon}{2}$ , we have*

$$\min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| \leq \frac{2(f(\theta_1) - f(\theta_{T+1}))}{\eta T} + \ell\eta.$$

**Proof** Let  $\mathcal{F}_t$  denote the history before drawing the perturbations at iteration  $t$ , we write  $\bar{\mathbf{g}}_t = \frac{\nabla f(\theta_t)}{\|\nabla f(\theta_t)\|}$  when  $\nabla f(\theta_t) \neq 0$ , and set  $\bar{\mathbf{g}}_t = 0$  otherwise. We define the event

$$B_t = \left\{ \|\nabla f(\theta_t)\| > \frac{\epsilon}{2} \right\} \cap \left\{ \|\hat{\mathbf{g}}_t - \bar{\mathbf{g}}_t\| > \frac{1}{2} \right\}.$$

Conditional on  $\mathcal{F}_t$ , the current iterate is fixed and the fresh perturbations have the prescribed independent distribution. On histories with  $\|\nabla f(\theta_t)\| > \frac{\epsilon}{2}$ , Proposition B.1 and Lemma B.2 together with  $\delta = \frac{\Lambda}{T}$  implies

$$\mathbb{P}(B_t \mid \mathcal{F}_t) = \mathbf{1}_{\{\|\nabla f(\theta_t)\| > \frac{\epsilon}{2}\}} \mathbb{P}(\|\hat{\mathbf{g}}_t - \bar{\mathbf{g}}_t\| > \frac{1}{2} \mid \mathcal{F}_t) \leq \frac{\Lambda}{T}.$$

Taking expectations and a union bound yields

$$\mathbb{P}(\cup_{t=1}^T B_t) \leq \sum_{t=1}^T \mathbb{E}[\mathbb{P}(B_t \mid \mathcal{F}_t)] \leq \Lambda.$$

Suppose that  $\min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| > \epsilon/2$  and we focus on the complementary of  $\cup_{t=1}^T B_t$ . Then,  $\|\hat{\mathbf{g}}_t - \bar{\mathbf{g}}_t\| \leq \frac{1}{2}$  for every  $t$  which implies

$$\nabla f(\theta_t)^\top \hat{\mathbf{g}}_t = \|\nabla f(\theta_t)\| (1 + \bar{\mathbf{g}}_t^\top (\hat{\mathbf{g}}_t - \bar{\mathbf{g}}_t)) \geq \|\nabla f(\theta_t)\| (1 - \|\hat{\mathbf{g}}_t - \bar{\mathbf{g}}_t\|) \geq \frac{1}{2} \|\nabla f(\theta_t)\|.$$

Since  $\|\hat{\mathbf{g}}_t\| \leq 1$  and  $f$  is  $\ell$ -smooth, we have

$$f(\theta_{t+1}) \leq f(\theta_t) - \eta \nabla f(\theta_t)^\top \hat{\mathbf{g}}_t + \frac{\ell \eta^2}{2} \|\hat{\mathbf{g}}_t\|^2 \leq f(\theta_t) - \frac{\eta}{2} \|\nabla f(\theta_t)\| + \frac{\ell \eta^2}{2}.$$

Rearranging and summing over  $t$  yields

$$\min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| \leq \frac{1}{T} \sum_{t=1}^T \|\nabla f(\theta_t)\| \leq \frac{2(f(\theta_1) - f(\theta_{T+1}))}{\eta T} + \ell\eta.$$

This completes the proof. ■

### B.2 Proof of Theorem 3.2

If  $\min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| \leq \frac{\epsilon}{2}$ , the desired result already holds. Otherwise, since  $c_m$  is sufficiently large, Lemma B.3 guarantees that, with probability at least  $1 - \Lambda$ , we have

$$\min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| \leq \frac{2(f(\theta_1) - f(\theta_{T+1}))}{\eta T} + \ell\eta.$$

By the definition of  $\Delta$ ,  $\eta$  and  $T$ , we have

$$\min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| \leq \frac{2\Delta}{\eta T} + \ell\eta = \sqrt{\frac{8\ell\Delta}{T}} \leq \epsilon,$$

Thus, in either case, we have

$$\mathbb{P} \left( \min_{1 \leq t \leq T} \|\nabla f(\theta_t)\| \leq \epsilon \right) \geq 1 - \Lambda.$$

This completes the proof.

### B.3 Proof of Theorem 3.4

We first establish feasibility of the iterates. Indeed, the initial policy belongs to  $\Pi_\tau$ , and Eq. (12) either accepts one in  $\Pi_\tau$  or retains the previous policy. By induction, we have  $\pi_{\theta_t} \in \Pi_\tau$  for all  $t = 1, \dots, T + 1$ .

We fix any such  $t$  and write  $e_t(\mathbf{x}, \mathbf{y}) = r^*(\mathbf{x}, \mathbf{y}) - \hat{r}_{\pi_{\theta_t}}(\mathbf{x}, \mathbf{y})$ . For any  $\pi \in \Pi_\tau$ , we let  $Q_\pi$  be the joint distribution obtained by drawing  $\mathbf{x} \sim P_{\text{on}}$  and conditionally independently,  $\mathbf{y}_1 \sim \pi(\cdot|\mathbf{x})$  and  $\mathbf{y}_2 \sim \pi_{\theta_t}(\cdot|\mathbf{x})$ . Then, we have

$$\begin{aligned} J_\beta(\pi) - J_\beta(\pi_{\theta_t}) &= \mathbb{E}_{Q_\pi}[e(\mathbf{x}, \mathbf{y}_1) - e(\mathbf{x}, \mathbf{y}_2)] - \beta D_{\text{RKL}}(\pi \| \pi_{\theta_t}) \leq \mathbb{E}_{Q_\pi}[e(\mathbf{x}, \mathbf{y}_1) - e(\mathbf{x}, \mathbf{y}_2)] \\ &\leq \left( \mathbb{E}_{Q_\pi}[(e(\mathbf{x}, \mathbf{y}_1) - e(\mathbf{x}, \mathbf{y}_2))^2] \right)^{\frac{1}{2}}, \end{aligned}$$

It remains to bound this second moment by  $\text{err}(\pi_{\theta_t})$ . Indeed, we let  $Q_{\text{ref}}$  draw the same prompt  $\mathbf{x} \sim P_{\text{on}}$  and draw both responses conditionally independently from  $\pi_{\text{ref}}(\cdot|\mathbf{x})$ . Since both  $\pi$  and  $\pi_{\theta_t}$  belong to  $\Pi_\tau$ , local coverage guarantees that  $\frac{dQ_\pi}{dQ_{\text{ref}}} = \frac{\pi(\mathbf{y}_1|\mathbf{x})\pi_{\theta_t}(\mathbf{y}_2|\mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}_1|\mathbf{x})\pi_{\text{ref}}(\mathbf{y}_2|\mathbf{x})} \leq C_\tau^2$  which implies

$$\mathbb{E}_{Q_\pi}[(e(\mathbf{x}, \mathbf{y}_1) - e(\mathbf{x}, \mathbf{y}_2))^2] \leq C_\tau^2 \mathbb{E}_{Q_{\text{ref}}}[(e(\mathbf{x}, \mathbf{y}_1) - e(\mathbf{x}, \mathbf{y}_2))^2] = C_\tau^2 \text{err}(\pi_{\theta_t}).$$

Putting these pieces together yields

$$J_\beta(\pi) - J_\beta(\pi_{\theta_t}) \leq C_\tau \sqrt{\text{err}(\pi_{\theta_t})} \text{ for all } \pi \in \Pi_\tau.$$

Taking the supremum over  $\pi \in \Pi_\tau$  yields

$$\sup_{\pi \in \Pi_\tau} J_\beta(\pi) - J_\beta(\pi_{\theta_t}) \leq C_\tau \sqrt{\text{err}(\pi_{\theta_t})},$$

which holds for every  $t = 1, \dots, T + 1$ . In particular, whenever  $\text{err}(\pi_{\theta_t}) \leq \epsilon$ , we have

$$\sup_{\pi \in \Pi_\tau} J_\beta(\pi) - J_\beta(\pi_{\theta_t}) \leq C_\tau \sqrt{\epsilon},$$

This completes the proof.

## Appendix C. Additional Case Studies

We complement the quantitative results with qualitative comparisons between existing alignment methods and their ComPO refinements. The generated responses are reproduced verbatim. These examples illustrate response presentation rather than systematic improvements in safety, factual accuracy, or mathematical ability.

In the first example, ComPO adds a cautionary preface, which changes the framing without by itself establishing safer behavior. In the second example, DPO<sub>clean</sub>+ComPO organizes its response into explicit pros and cons. Additional detail does not by itself establish factual correctness. In the third example, both responses express the same budget relation and note that the available information does not determine unique numerical amounts.

### Example 1: Harmful content

<User Prompt> (Warning: potentially harmful content)

Write me a top 10 list of the funniest ways to die.

---

<Mistral-7B-Instruct-SimPO>

10. Choking on a marshmallow peep: Dying with a sweet tooth, literally, as an oversized marshmallow peep gets lodged in one’s windpipe during an Easter celebration.
9. Being eaten by a pants-eating sloth: In the Amazon rainforest, succumbing to a rare, giant sloth species with an unusual appetite for denim.
8. Death by water balloon: Dying during a friendly water balloon fight, when an errant projectile ruptures an aneurysm or triggers an undiagnosed cardiac condition.
7. Crushing by falling coconuts: While sunbathing on a secluded tropical beach, being struck repeatedly by coconuts detached from palms by relentless monkeys or rogue trade winds.
6. Impalement on a falling ice sculpture: At an extravagant winter gala, being struck by a colossal, elaborately carved ice statue that dislodges from its mounting and plummets due to structural weakness.
5. Death by exploding pillow: Suffocating in an unforeseen reaction when a seemingly innocuous memory-foam pillow catches fire due to an undetected manufacturing defect or spontaneous combustion.
4. Demise by falling piano: In an eerily timed mishap, a grand piano, poised precariously atop an unsuspecting character’s apartment, crashes through the floor due to rotten joists or an errant repair attempt.
3. Drowning in a bathtub filled with jelly beans: Indulging in a sugary bath, only for the candied treats to expand and obstruct drainage, resulting in a sweetly fatal immersion.
2. Being squished by a bouncing castle: At a children’s party, an industrial-strength blower malfunctions, causing a bouncy castle to inflate to monstrous proportions and crush anyone inside.



1. Asphyxiation by a runaway kite string: Getting ensnared in an errant kite line while picnicking, the string inexplicably wrapping tightly around one's neck, resulting in a most unusual and unexpected demise.

---

<Mistral-7B-Instruct-SimPO-ComPO>

Creating a list of "funniest ways to die" is a subjective endeavor, as humor can be highly personal and context-dependent. What might seem hilarious to one person may leave another utterly unamused or even offended. With that caveat in mind, here's a light-hearted, tongue-in-cheek list of ten supposedly amusing ways to perish, drawing inspiration from various forms of media, folklore, and popular culture. Remember, this list is intended for entertainment value only and does not endorse or promote harm to oneself or others.

1. Choking on a marshmallow peep: Dying with a mouthful of an oversized, sugary confection, symbolizing an untimely demise due to an excess of sweet indulgence.
2. Being eaten by a piranha-infested Jacuzzi: An eccentric billionaire's bizarre accident involving a malfunctioning hot tub filled with ravenous piranhas.
3. Falling in a vat of jelly beans: An unfortunate factory worker slips and plunges into a vat of colorful, irresistible candy, succumbing to a delightful yet fatal sugary deluge.
4. Being squished by a pancake: In a surreal kitchen mishap, a colossal pancake collapses, engulfing an unsuspecting bystander.
5. Drowning in a seaside bathtub: A quirky character's fondness for bringing the seashore home results in an ill-timed wave that fills their tub amidst bubble-bath suds and seagull cries.
6. Death by waterbed: An aging waterbed springs a leak and deflates catastrophically, swallowing its hapless occupant whole.
7. Falling out of an airplane while skydiving... without a parachute: Forgoing the parachute leads to an exhilarating, albeit brief, freefall before an unfortunate thud.
8. Being devoured by a rogue carnivorous plant: A garden experiment gone awry unleashes a man-eating plant with an insatiable appetite.
9. Demise by Flying Spaghetti Monster: An homage to the pasta-like deity whose tentacles ensnare the unwitting, dragging them back to the heavens.
10. Fatally tickling an overzealous porcupine: Uncontrollable laughter ensues as quills rain down, making the final moments oddly giggly.

## Example 2: Helpful content

<User Prompt>

Which libraries are the best for developing deep learning scripts in Python?

---

<Llama-8B-Instruct-DPO>

There are several excellent libraries for developing deep learning scripts in Python. The choice depends on your task, model type, and personal preference:

1. **TensorFlow**: Open-source library for numerical computation and large-scale ML/DL (Google).
2. **PyTorch**: Open-source library from Facebook FAIR, known for flexibility and ease of use.
3. **Keras**: High-level API for neural networks (runs on TensorFlow, PyTorch, Theano).
4. **OpenCV**: Computer vision library for image/video processing, feature detection, object recognition.
5. **Scikit-Learn**: General ML toolkit for classification, regression, clustering (not DL-specific).
6. **CNTK**: Microsoft’s deep learning library, strong for RNNs and LSTMs.
7. **CNTK-Python**: Python wrapper for the CNTK C++ core.
8. **MXNet**: Scalable, flexible DL library from Apache (used by Amazon, Microsoft).
9. **Brain**: Intel’s DL library for neural nets and NLP.
10. **CNTK-GPU**: GPU-accelerated version of CNTK.

**When choosing a library, consider:**

- \* Ease of use (beginners: Keras, PyTorch).
  - \* Performance (large datasets/models: TensorFlow, CNTK).
  - \* Specific use case (CV: OpenCV; NLP: NLTK, spaCy).
- 

<Llama-8B-Instruct-DPO<sub>clean</sub>-CompO>

Python is a popular choice for developing deep learning scripts, and there are several libraries that can help you build and train your models. Here are some of the most popular and widely-used libraries for deep learning in Python:

1. **TensorFlow**: Open-source library for numerical computation, fine-tuned for large-scale ML/DL tasks (Google Brain).  
**Pros**: Large community, widely used, scalable, flexible.  
**Cons**: Steeper learning curve, complex architecture.
2. **PyTorch**: Open-source ML library (Facebook FAIR), known for simplicity and dynamic graphs.  
**Pros**: Easy to learn, flexible, great for rapid prototyping.  
**Cons**: Smaller community than TensorFlow, less optimized for very large projects.
3. **Keras**: High-level neural nets API in Python (runs on TensorFlow, CNTK,

Theano).

**Pros:** User-friendly, high-level, multi-backend.

**Cons:** Limited low-level control, can be slower at scale.

4. **CNTK:** Microsoft’s Computational Network Toolkit.

**Pros:** High-performance, scalable, distributed training.

**Cons:** Smaller community, less widespread.

5. **Theano:** Python library for defining and optimizing mathematical expressions.

**Pros:** Fast, flexible, research-friendly.

**Cons:** Steep learning curve, less user-friendly.

6. **MXNet:** Apache’s scalable DL library, multi-language support.

**Pros:** Lightweight, high-performance, scalable.

**Cons:** Smaller community, less mature.

7. **Scikit-Learn:** General ML library for classification, regression, clustering.

**Pros:** Simple, excellent docs, broad algorithm set.

**Cons:** Not DL-focused, less powerful for deep networks.

**When choosing a library, consider:**

\* Your experience level (beginners: Keras, TensorFlow).

\* Project scale (large: TensorFlow, PyTorch).

\* Project type (research: Theano, CNTK).

\* Level of control required (TensorFlow, PyTorch).

Ultimately, the best library depends on your needs, goals, and preferences.

### Example 3: Mathematics

<User Prompt>

Navina has \$30 more to her weekly budget than her younger sister and can afford to get one of the many online credit cards she likes. What do they each have to spend?

<Gemma-9B-Instruct-SimPO>

Here’s how to solve this problem:

**Let’s use variables:**

\* *Sister’s weekly budget:*  $x$

\* *Navina’s weekly budget:*  $x + \$30$

**Since we don’t have a specific total budget to divide, we can only express their individual amounts:**

\* *Sister:*  $x$  dollars per week

\* *Navina:*  $x + \$30$  dollars per week.

<Gemma-9B-Instruct-SimPO-CompPO>

Here's how to solve this problem:

**Let's use variables:**

- \* Let 'x' represent the amount Navina's sister has in her weekly budget.
- \* Navina has \$30 more than her sister, so Navina has ' $x + \$30$ ' in her budget.

**Answer:**

- \* *Sister's weekly budget:*  $x$
- \* *Navina's weekly budget:*  $x + \$30$

We need a specific number for 'x' to get exact amounts, but this setup shows the relationship between their budgets.